

RumiCarハンズオン！

自動運転アルゴリズムに挑戦しよう

@北とぴあ
2026年08月11日

講師：千葉 瑠実佳
RumiCar開発部
プロジェクトオーナー



RumiCar 開発部 紹介



千葉 瑠実佳



千葉 吉輝



片岡 秀明



小野寺 修



石垣 翔子



稲玉 繁樹

目次

- 学べる内容
- RumiCarとは
- 準備
- 距離の測定
 - Exercise-1.1 中央のセンサで測距しよう！
 - Exercise-1.2 3つのセンサで測距だ
 - Exercise-1.3 シリアルプロッタを使ってみよう
- モータ制御
 - Exercise-2.1 ハンドルを切る
 - Exercise-2.2 速度制御
 - Exercise-2.3 前進と後進
 - Exercise-2.4 ジグザグ走行
- 自律走行基礎
 - Exercise-3.1 安全に停止する車
 - Exercise-3.2 市街地を走る車

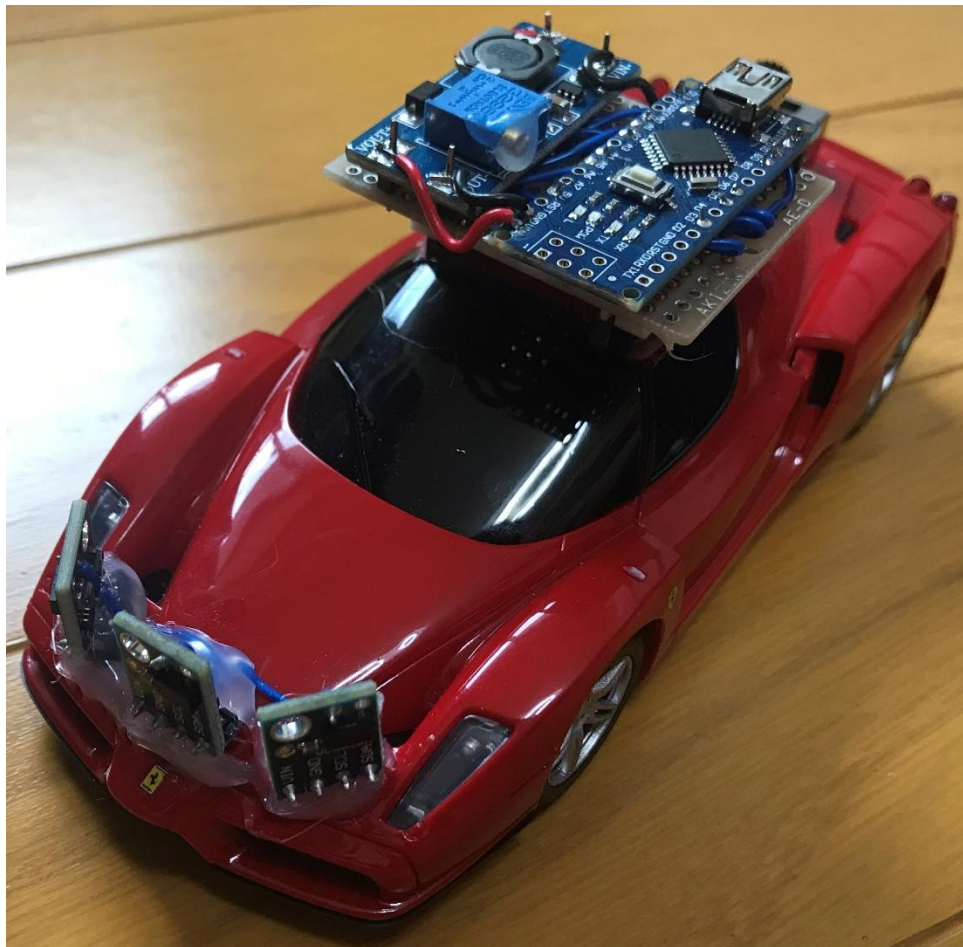
青字のところで
実際にプログラ
ミングするよ

学べる内容

- RumiCarの仕組み
 - センサー
 - モータドライバ
- レーザー測距モジュールを使った距離測定
- モータドライバ制御
- 自律走行基礎

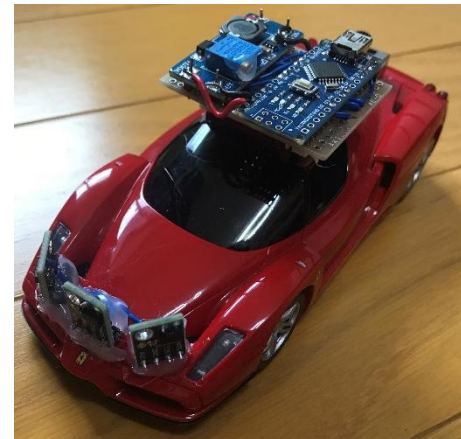
- その他として、良いプログラム開発のための関数化の意義、なども学べます

RumiCarとは

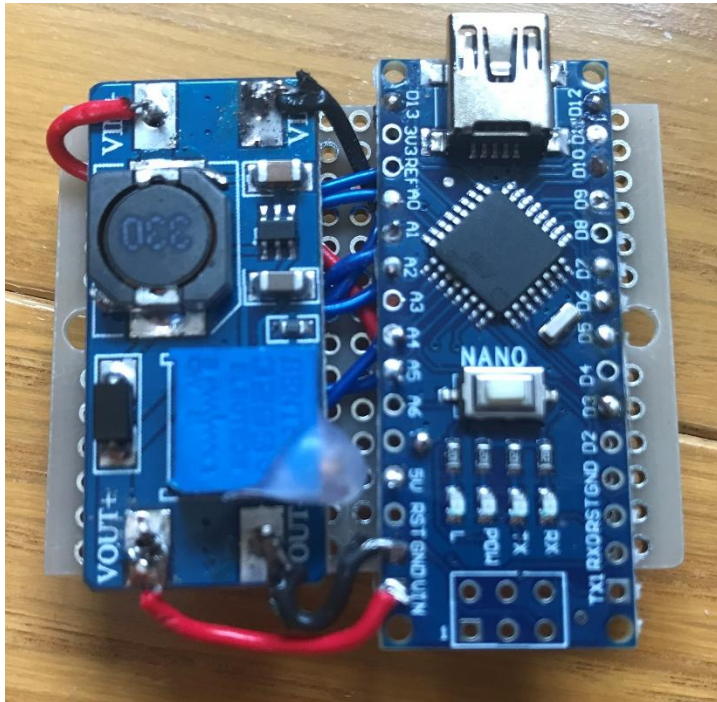


RumiCarとは

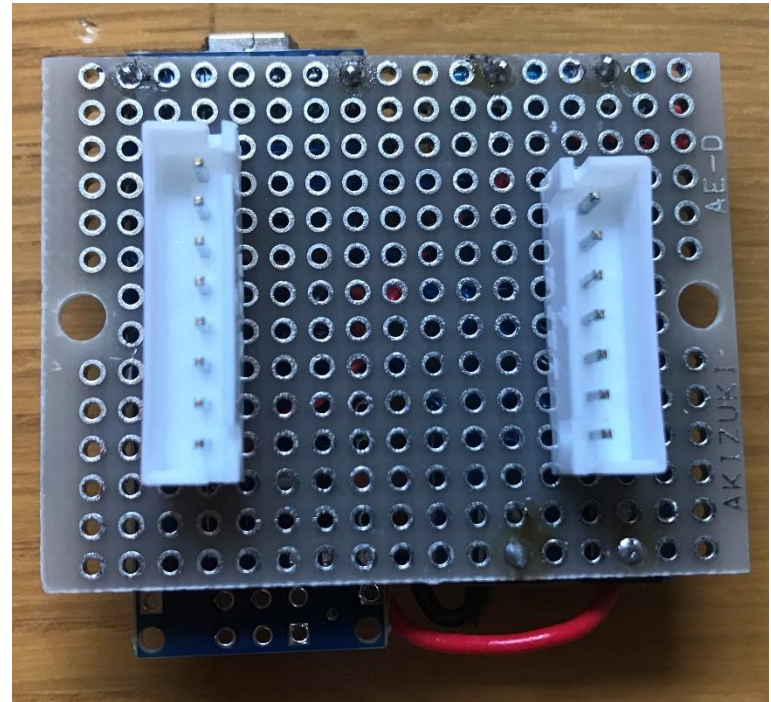
- 複数の車体と多様な種類のコンピュータモジュールを組み合わせて自律走行プログラムを開発可能なプラットフォームです
- レーザー測距モジュールを搭載していて障害物迄の距離を計測することができます
- モータの回転を制御することにより車速制御できます



コンピューターモジュール (CM)

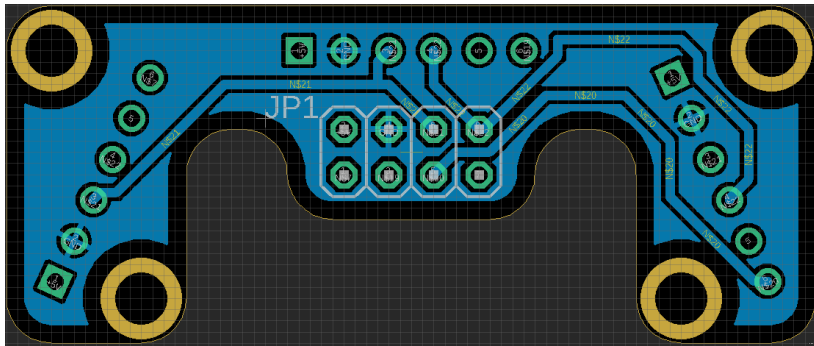


表面



裏面

コンピューターモジュール (CM)基板化

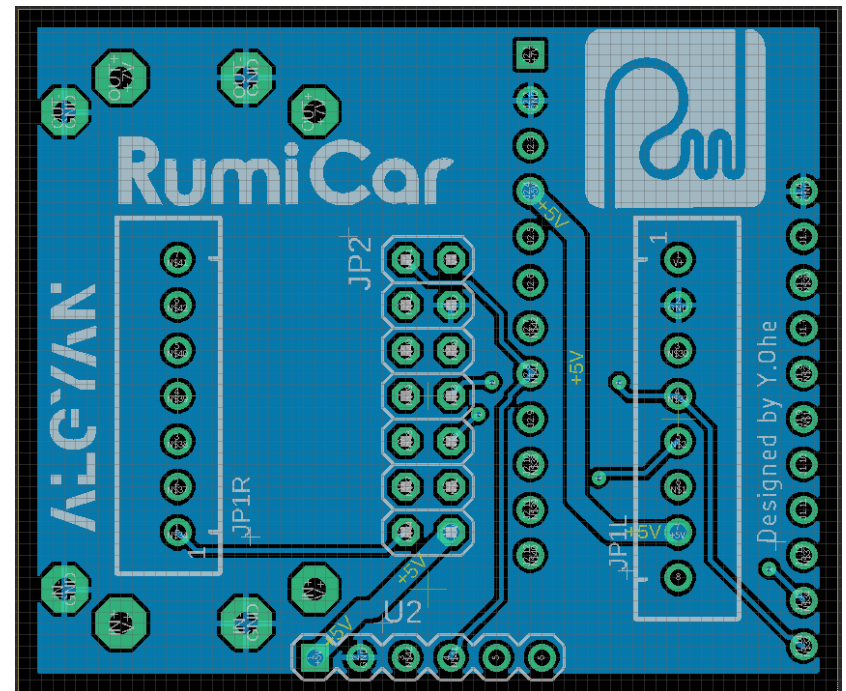


センサー基板

開発・設計してくださったのは
大栄 豊 氏



SeeedのFusion PCBに発注。
納品された。



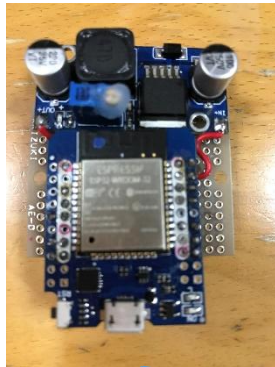
コンピューターモジュール基板

組み合わせ自由自在

Arduino Nano



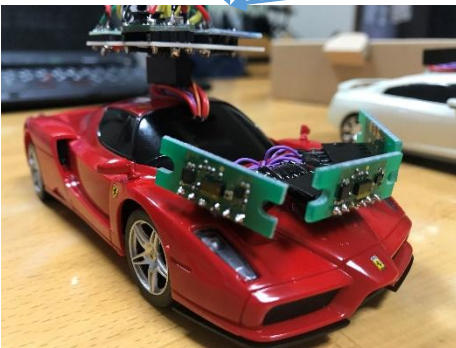
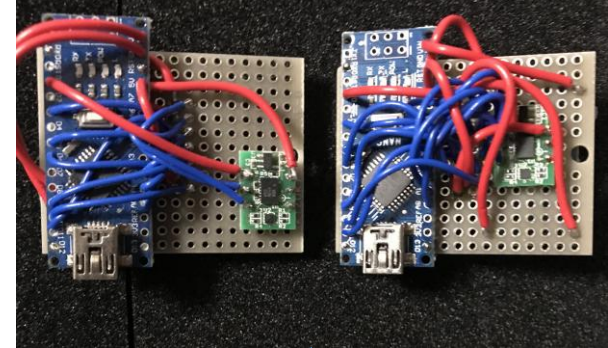
ESP32



ESP32+EasyVR3



Arduino Nano+BMX055



組み合わせ自由自在

次なる取り組み(挑戦)

Raspberry Pi

- コンピューターモジュール化
- モーター制御
- 画像認識

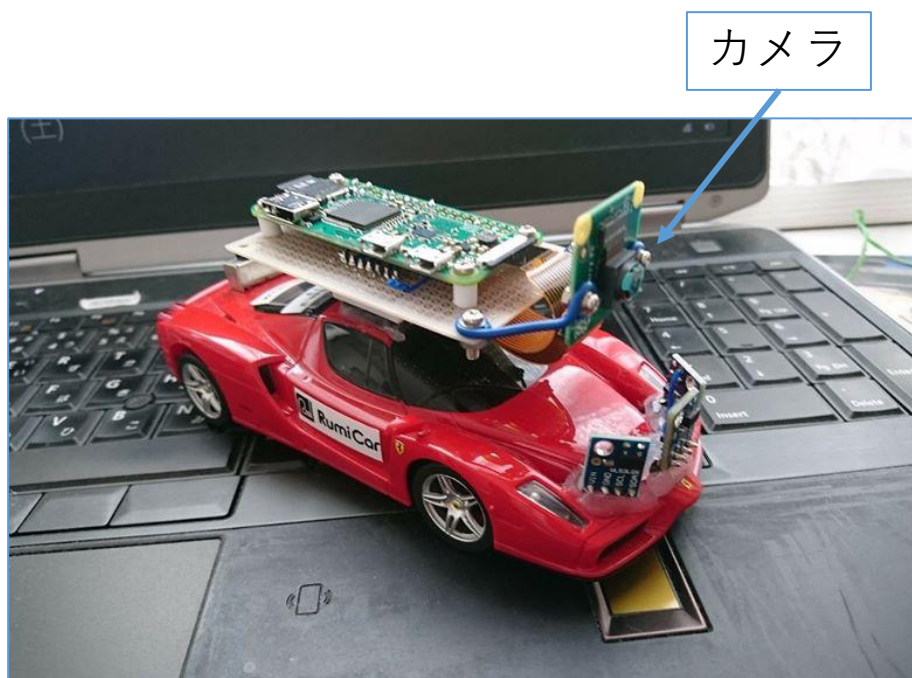


Pythonでコードを書き、
モーター制御などしてくださる
石垣 翔子 氏

ラズパイ版CM（カメラ付き）

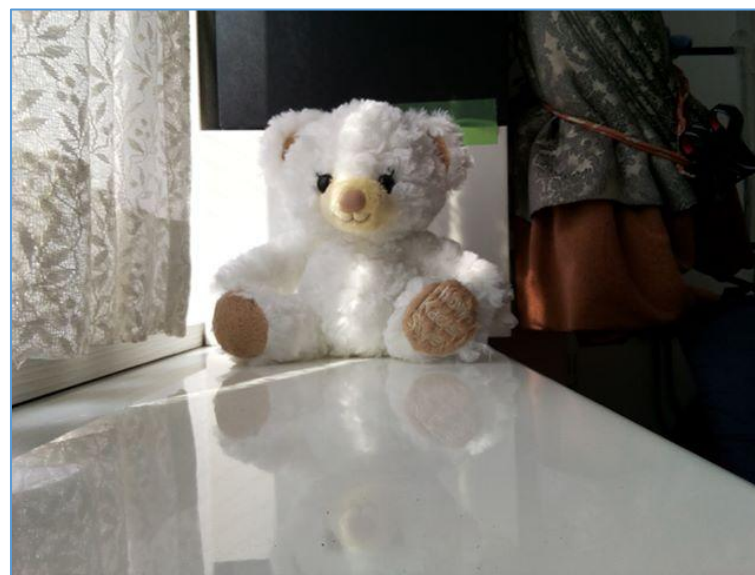


ラズパイ版CM（カメラ付き）



カメラ

ラズパイ版CMをRumiCarに装着



ラズパイ版CMのカメラで撮影

Obniz版CM

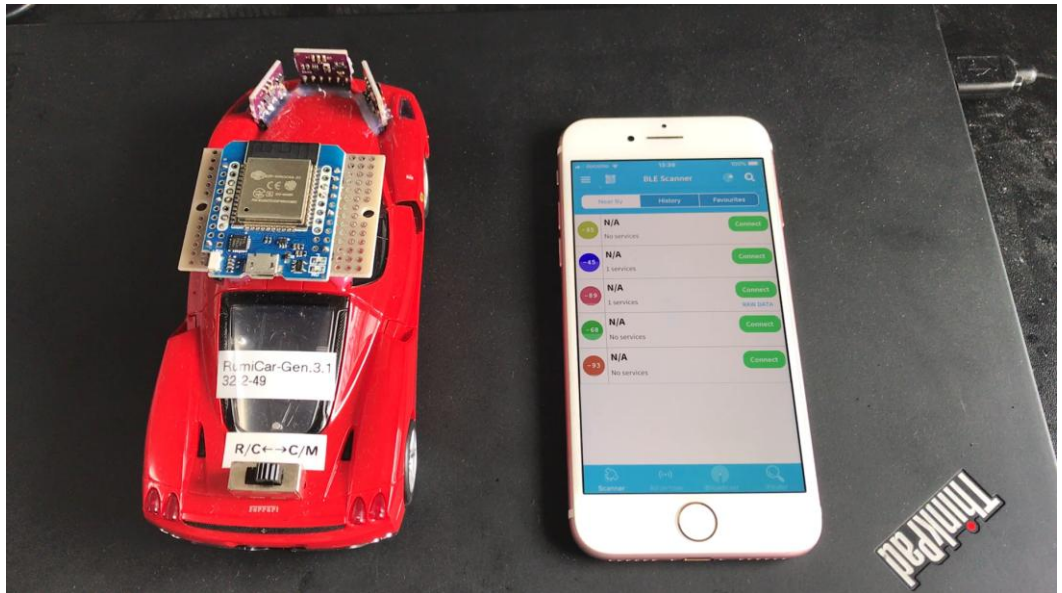
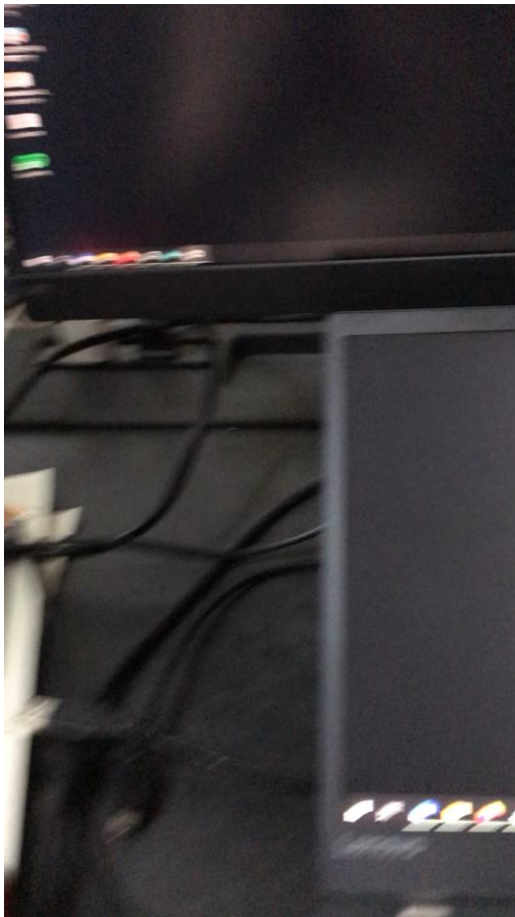


左側がObniz版、右側はラズパイ版

ESP32版 Bluetooth接続 (動画)

<https://youtu.be/-r6Vrhn5ifQ>

<https://youtu.be/UpEcTDvwYFw>



BLE(Bluetooth Low Energy)マルチコネクト

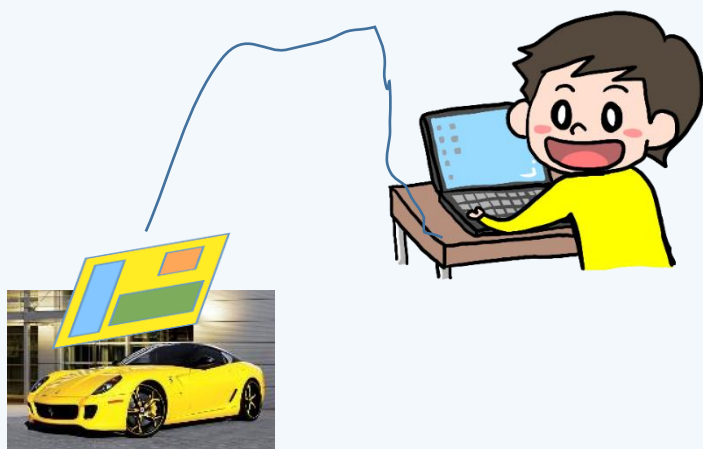
Bluetoothシリアル接続

コンピュータモジュール比較

ボード	Arduino Nano	ESP32	Raspberry Pi Zero W	Obniz	Spresense
電源	7-12V(VIN) (5Vラインへの供給 は非推奨)	2.2-3.6V(3V3へ入力の場合) 3.7-6.0V(VINへ入力の場合)	5V	5V	3.6-4.4V
内部動作電圧	5V	3.3V	3.3V	3.3V	
I/Oピン電圧	5V	3.3V	3.3V	5V(電圧選択 可能)	5V、3.3V切替
I2C電圧					1.8V
外部電源供給能力	ピン当たり40mA 全体で100mA	最大12mA	ピン当たり16mA 全体で50mA	5Vモードで 1A	6mA
消費電力(電流)	65mA	WiFi時160-260mA	125mA WiFi時270mA	平均170mA (WiFi不使用 時)	
PWMピン数	6本	4本	ハード2本	6本	
電源関係構成	電源:MT3608 VL/DRV: 5 Vラインより	PQ3RD23 (またはLM2596)	電源:MT3608 (またはTPS63020) VL/DRV:PQ3RD23 (またはSOT-23AP7333)	MT3608 (または TPS63020)	

プログラミングで楽しめるよ

自分でプログラミング
できるよ！



- RumiCarがあれば自分のパソコンでRumiCarへプログラムできるよ、そして自由に走らせよう
- AIに挑戦しよう
- みんながアッと驚くプログラムを作ってみよう！

世界の友達と交流しよう！

予定・計画中です

プログラムのダウンロードや交換が
できるよ

- 自分のプログラムをサーバーにアップしよう
- おもしろそうなプログラムを見つけたら自分のRumiCarにダウンロードして走らせてみよう
- 世界の友達と交流しよう



みんなで学ぼう！

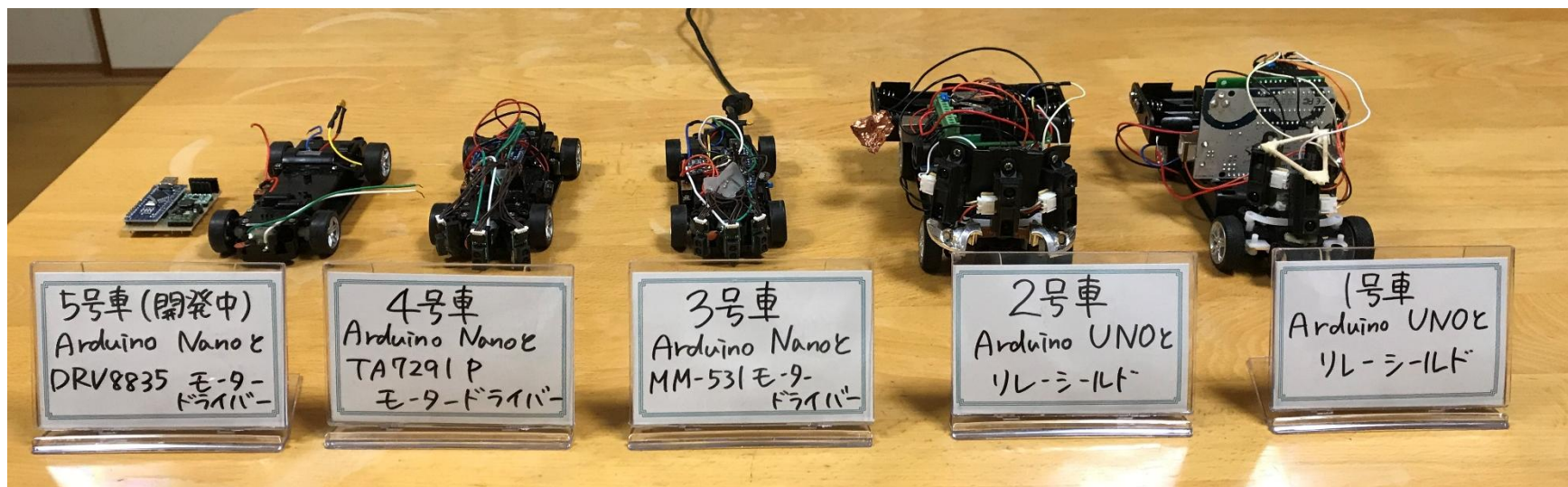
予定・計画中です

教室を開催するよ



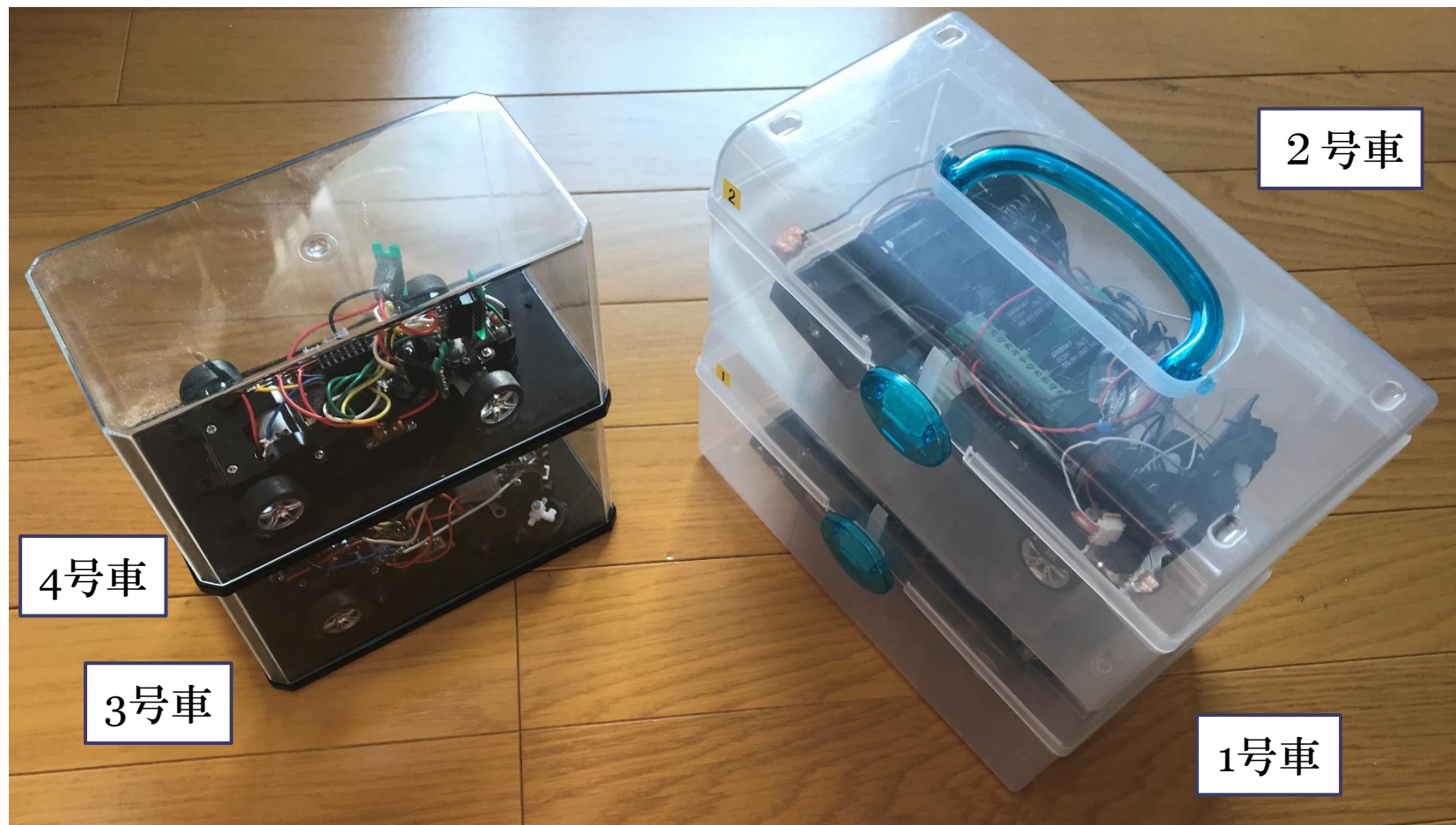
- RumiCarを作ろう！
- コンピュータモジュールを作ろう！
- プログラムを作ろう！

歴代RumiCar(1号車～5号車)

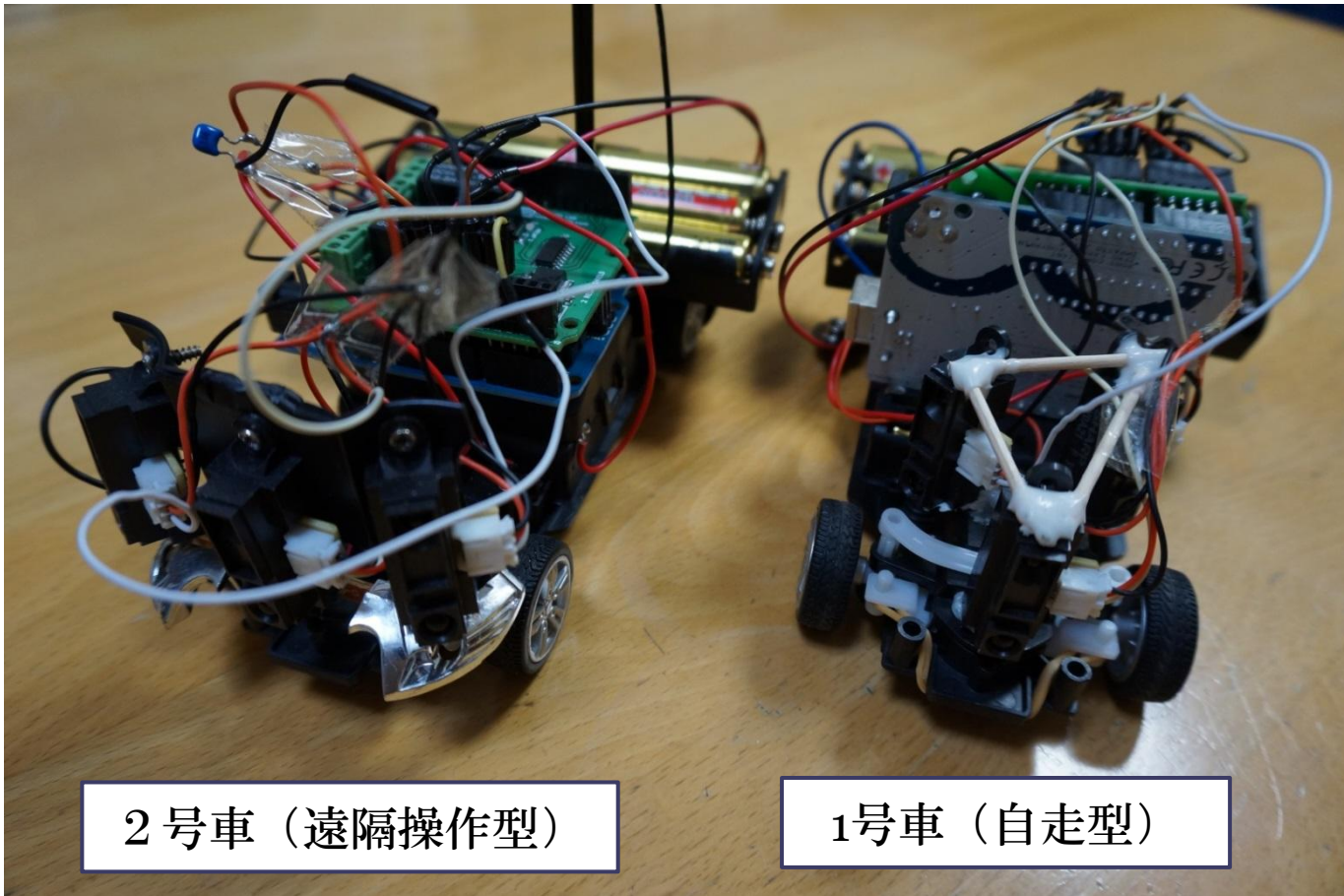


2017年8月3日撮影

RumiCarの歴史(博物館)



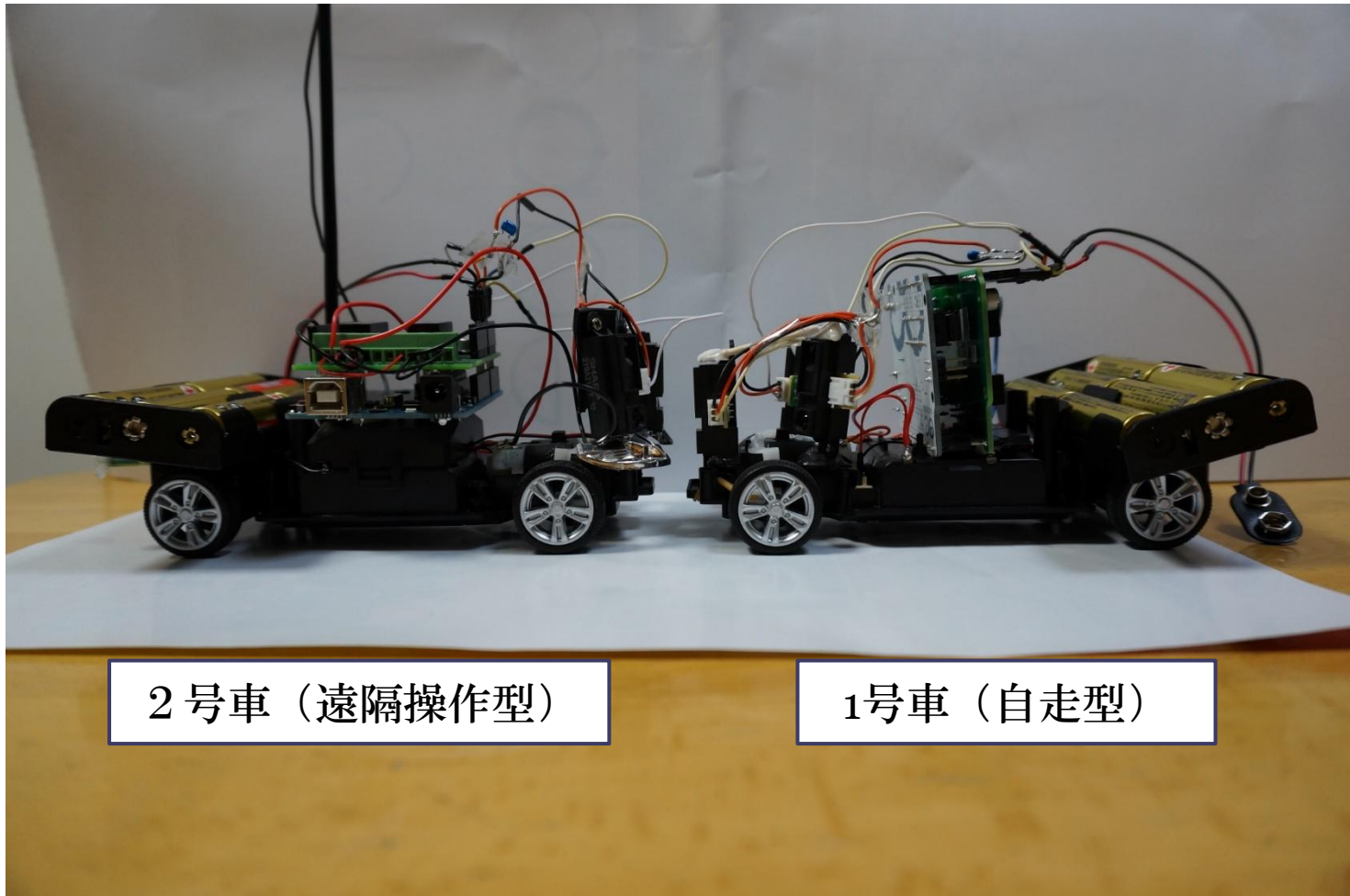
RumiCarの歴史(1号車と2号車)



2号車 (遠隔操作型)

1号車 (自走型)

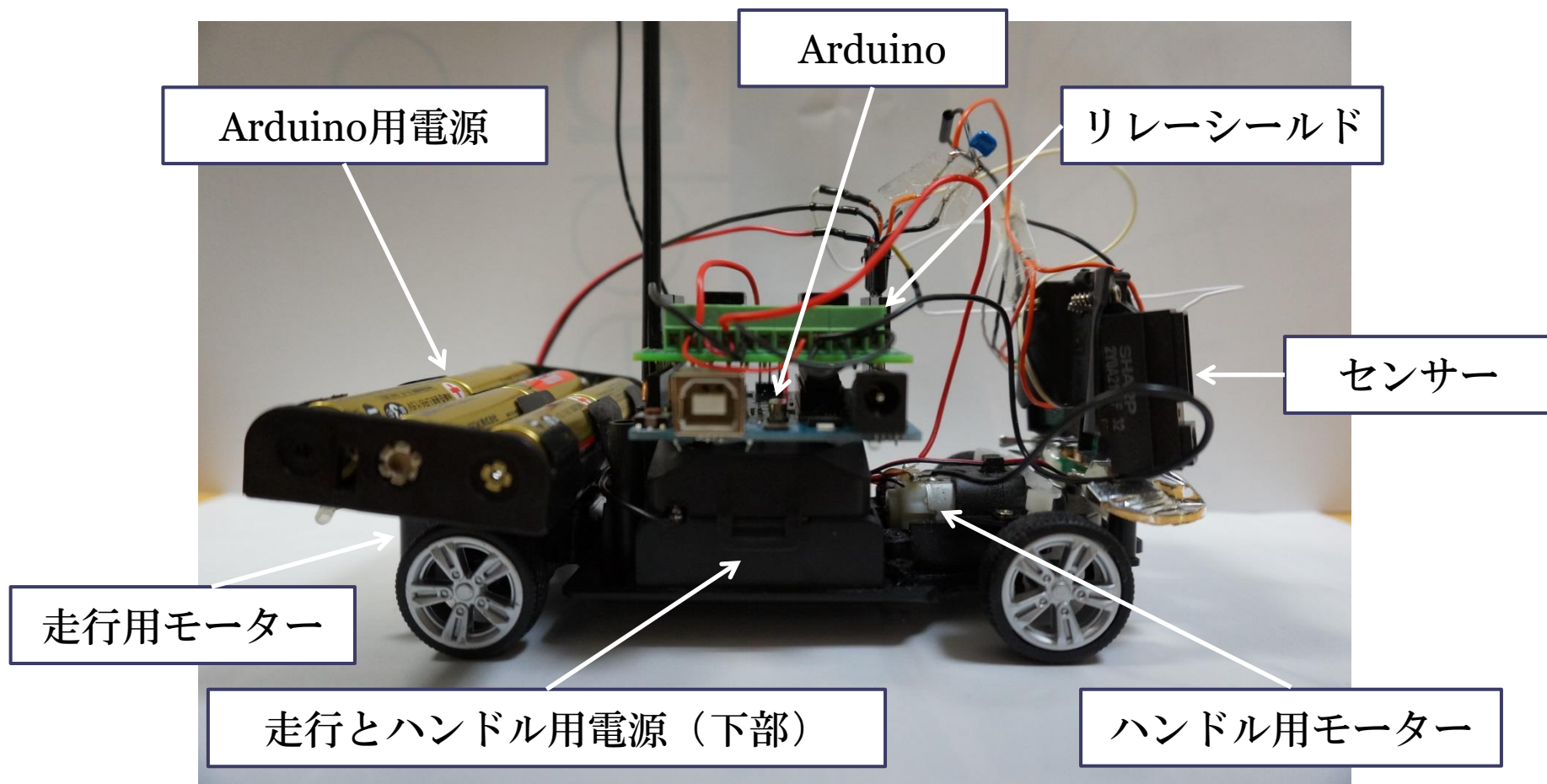
RumiCarの歴史(1号車と2号車)



2号車 (遠隔操作型)

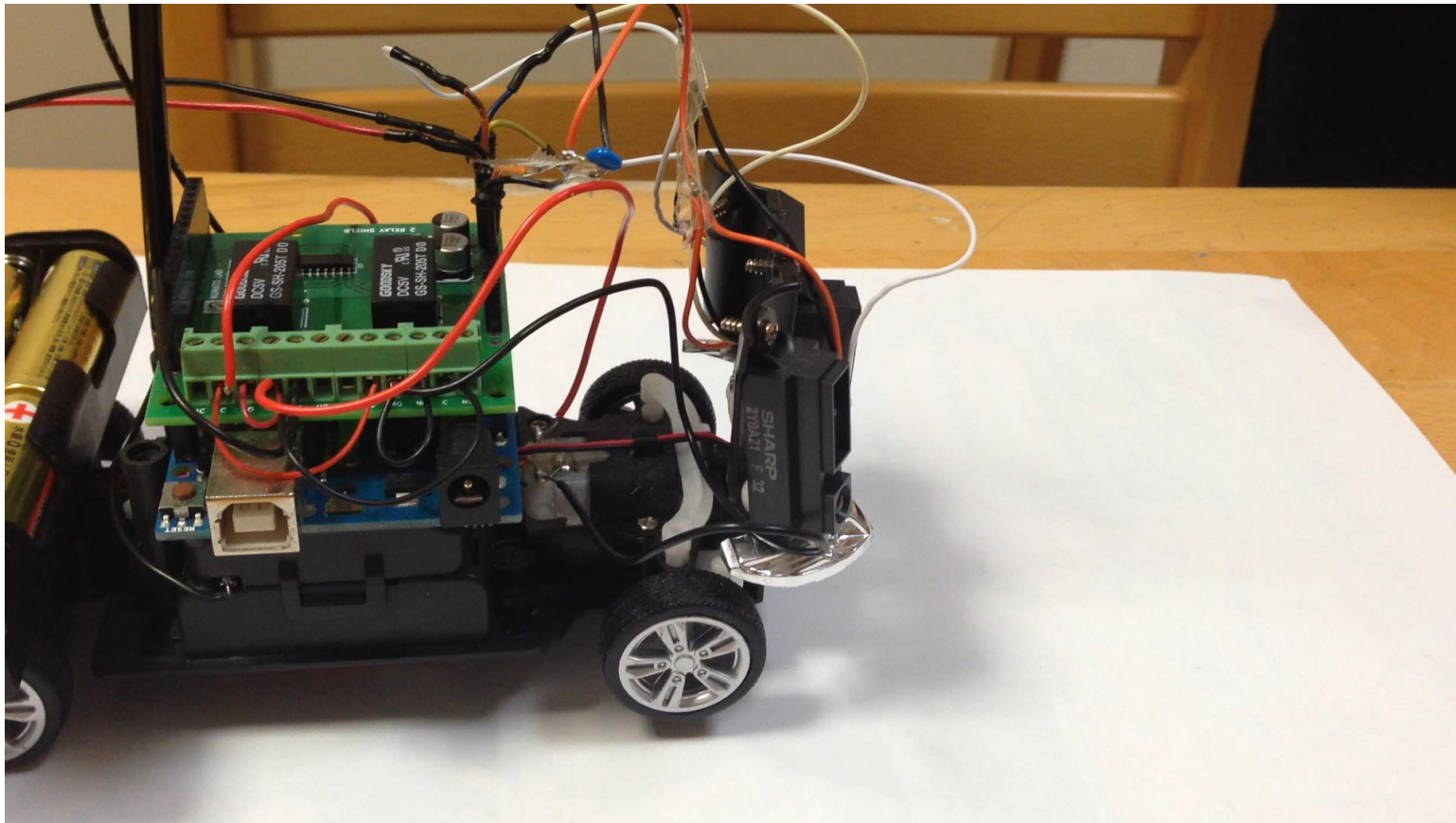
1号車 (自走型)

2号車の説明

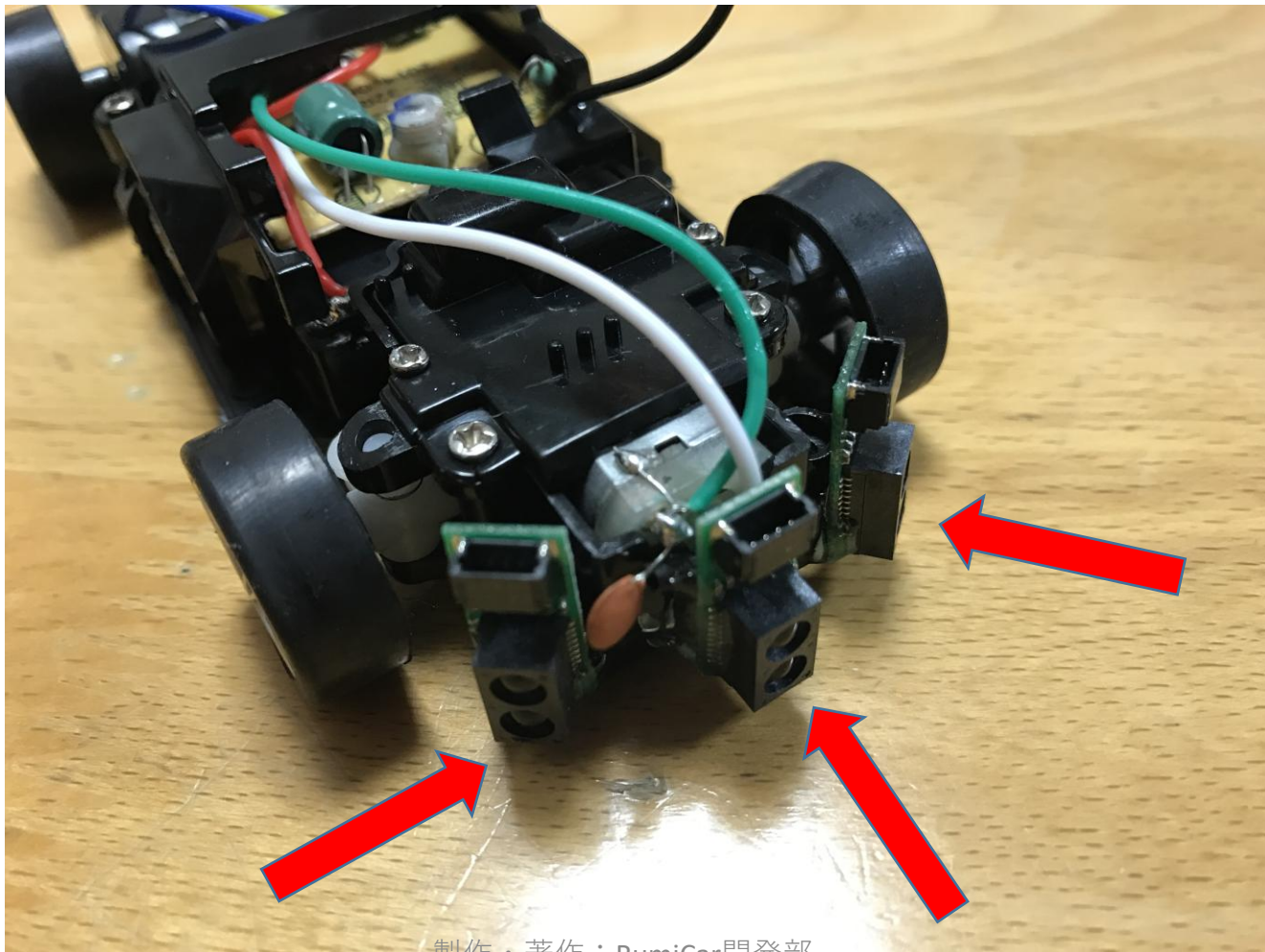


センサー動作試験（動画）

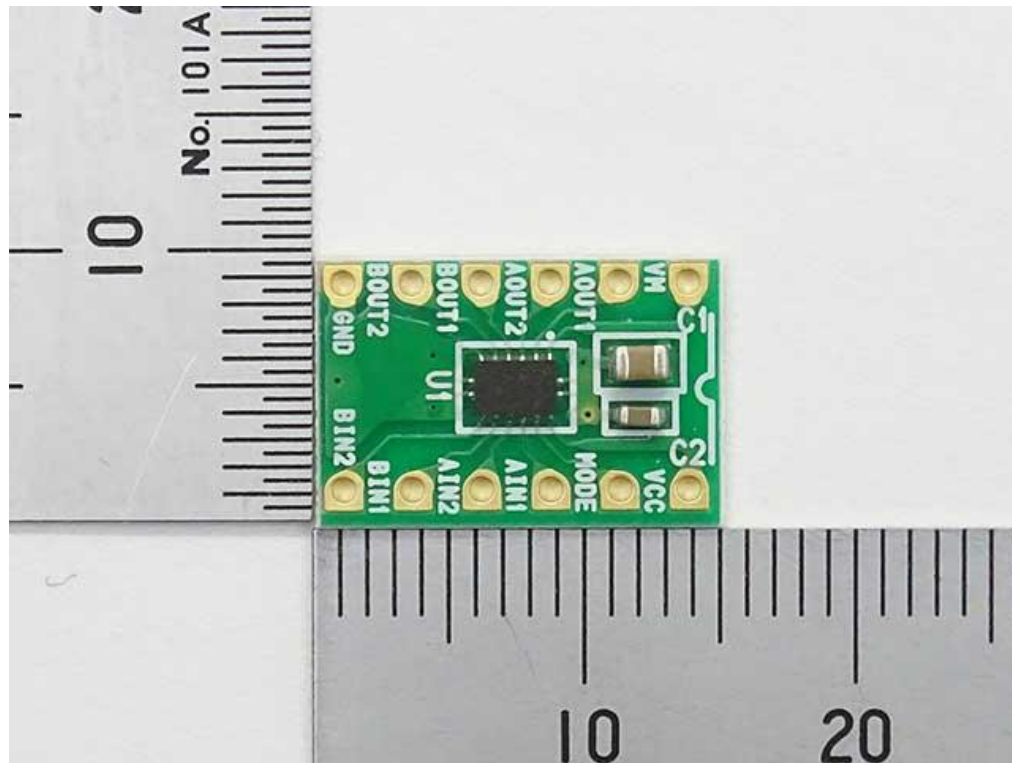
<https://youtu.be/vIAL7uUnXTA>



3、4号車センサー部分



モータドライバ



DRV8835

DRV8835 秋月電子通商より

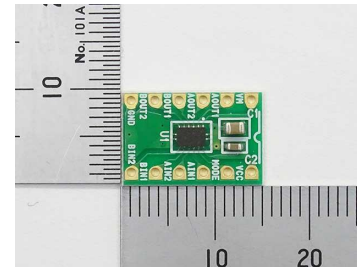
モータドライバ

- RumiCarに搭載されているモータドライバは DRV8835 です

DRV8835

- DRV8835の特徴

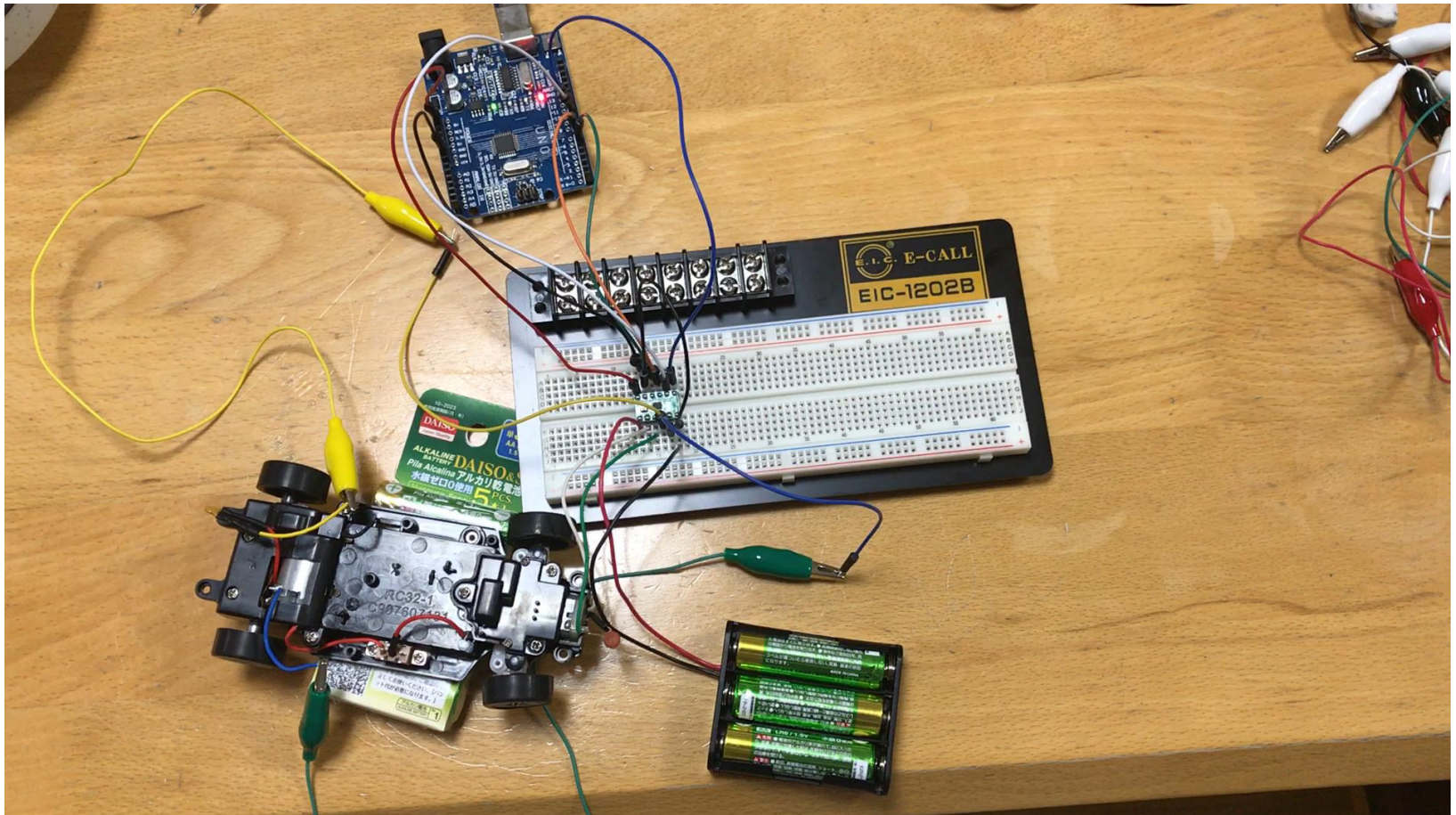
- 2チャンネル
- 各チャンネル1.5Aまで流せる
- PWM(250kHz)
- 低消費電力
- 低電圧降下（モータ側）
- ロジック電源（VCC）とモータ電源が独立している
 - ロジック電源電圧は2から7V
 - モータ電源電圧は11Vまで



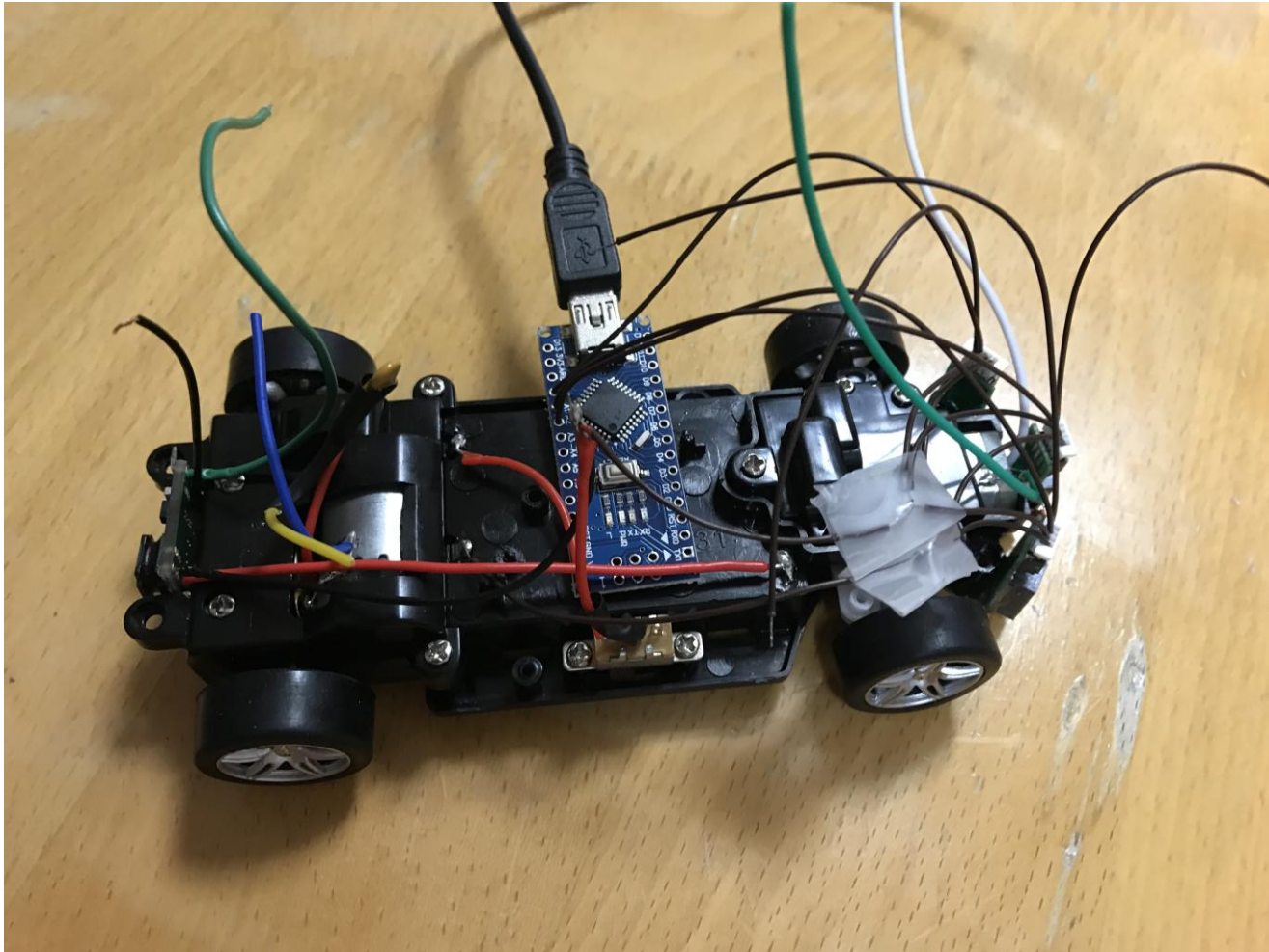
DRV8835 秋月電子通商より

開発中動画（動画）

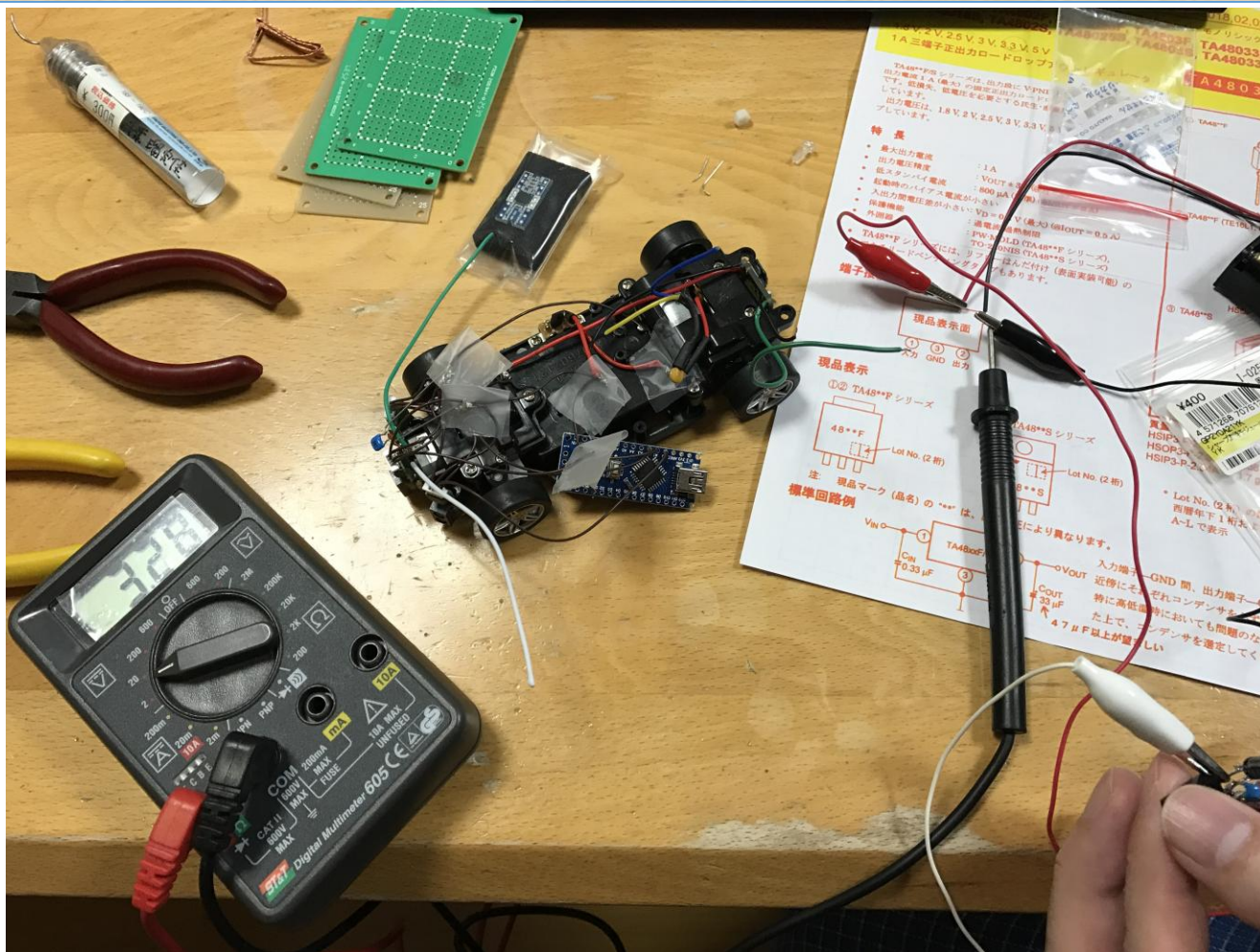
<https://youtu.be/8jULg3fZ-jw>



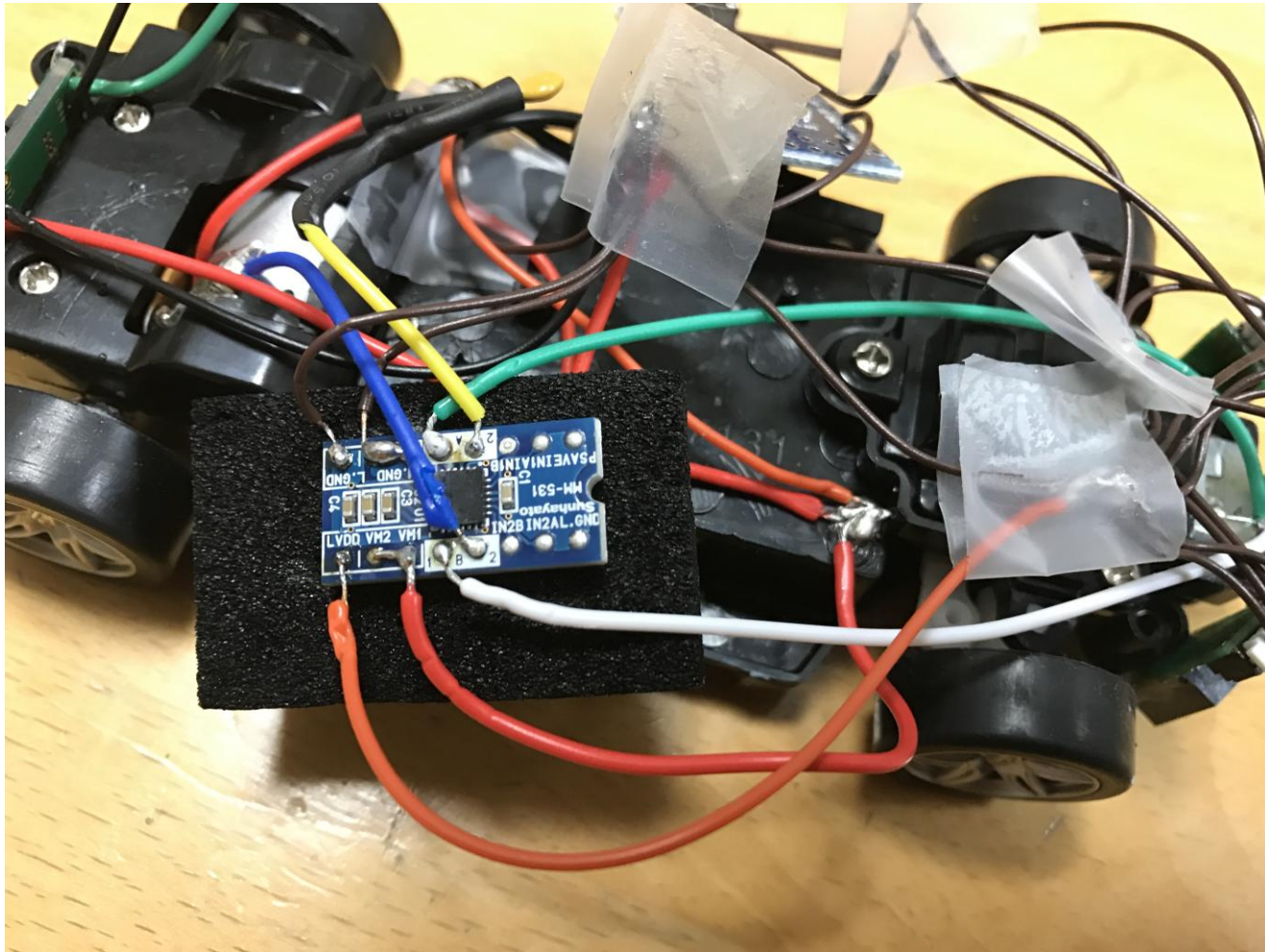
開発中写真



開発中写真

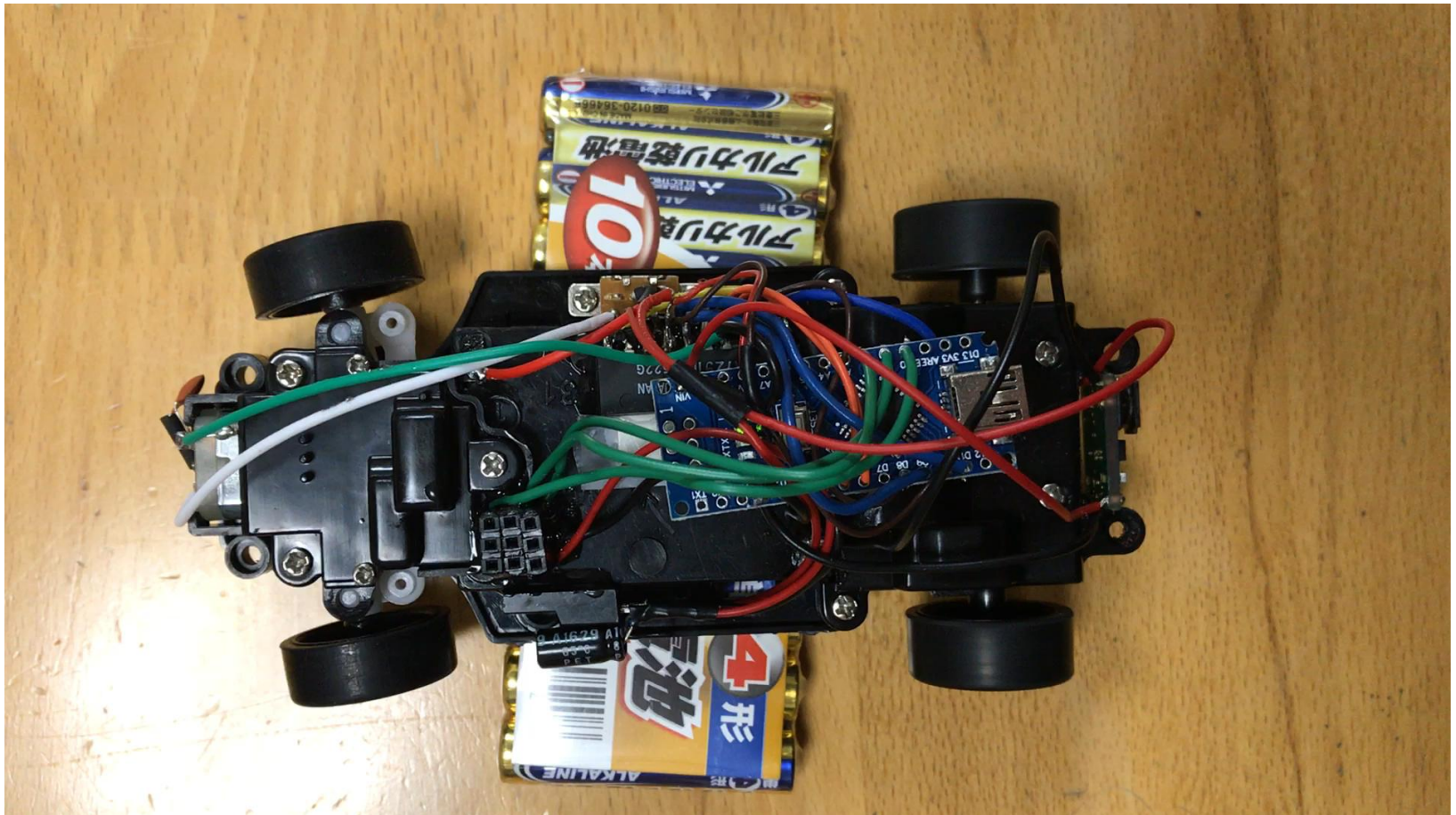


開発中写真



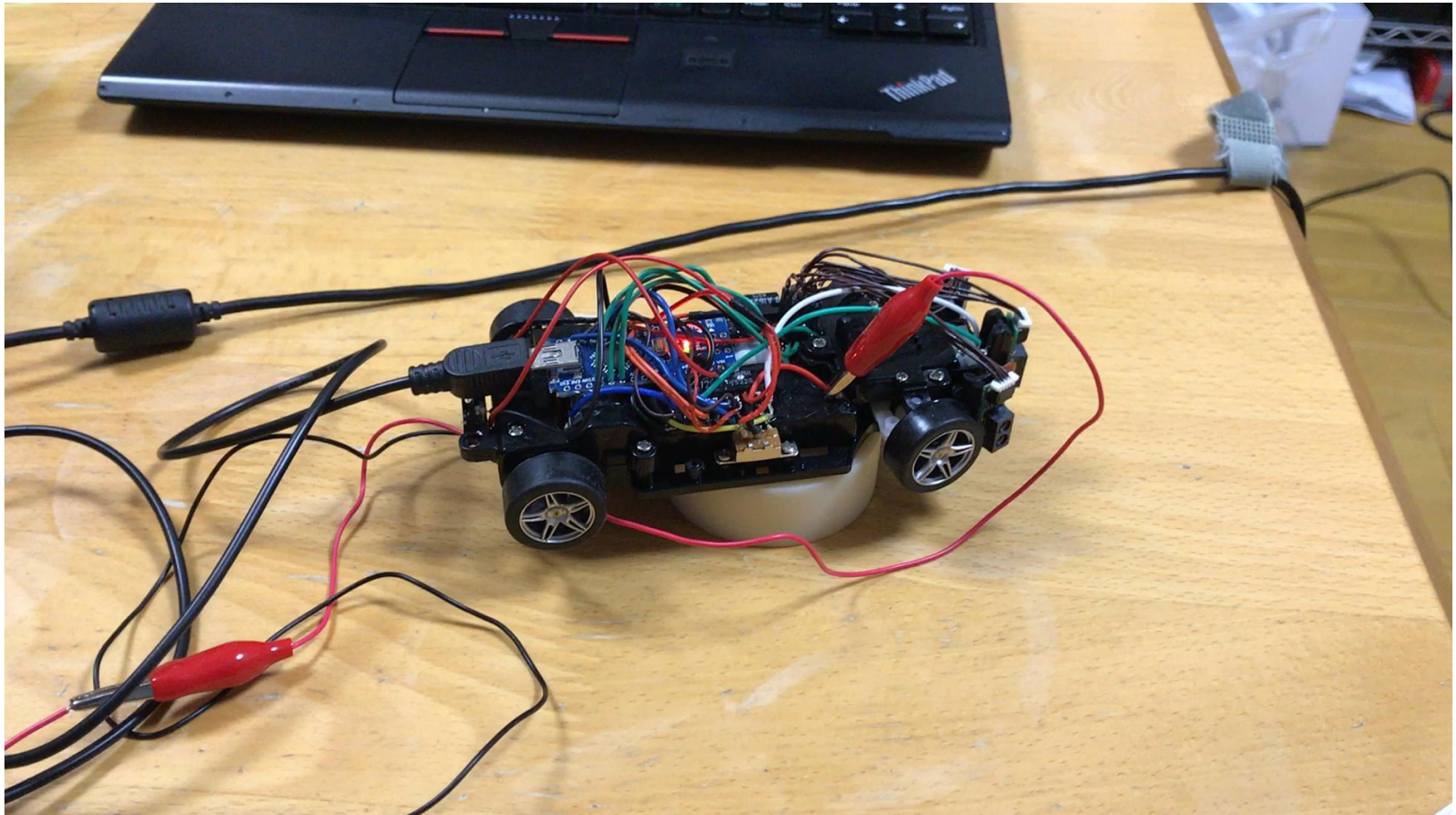
動作試験 (動画)

<https://youtu.be/8jULg3fZ-jw>



動作試験 (動画)

<https://youtu.be/8jULg3fZ-jw>

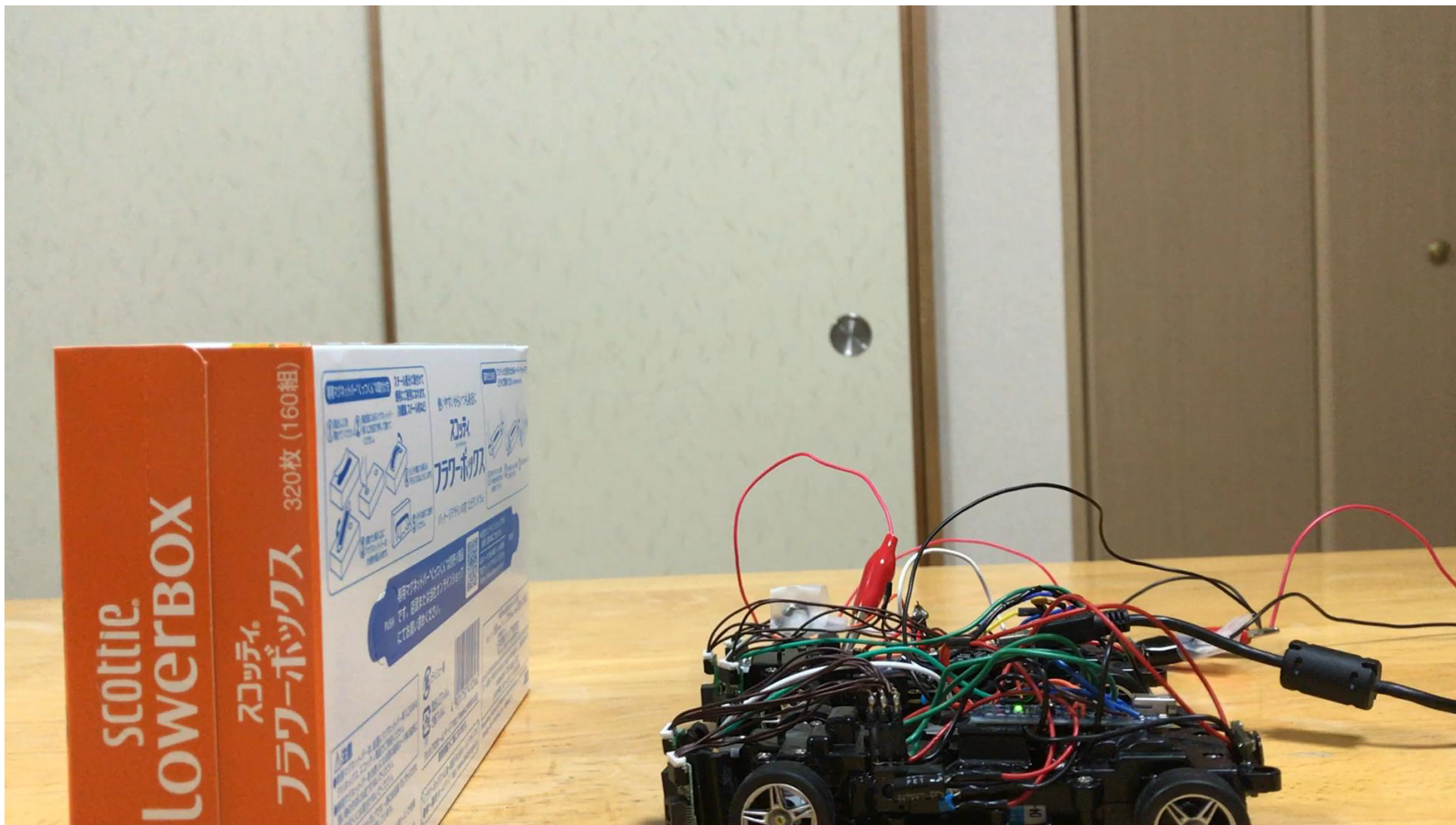


4号車システム部分

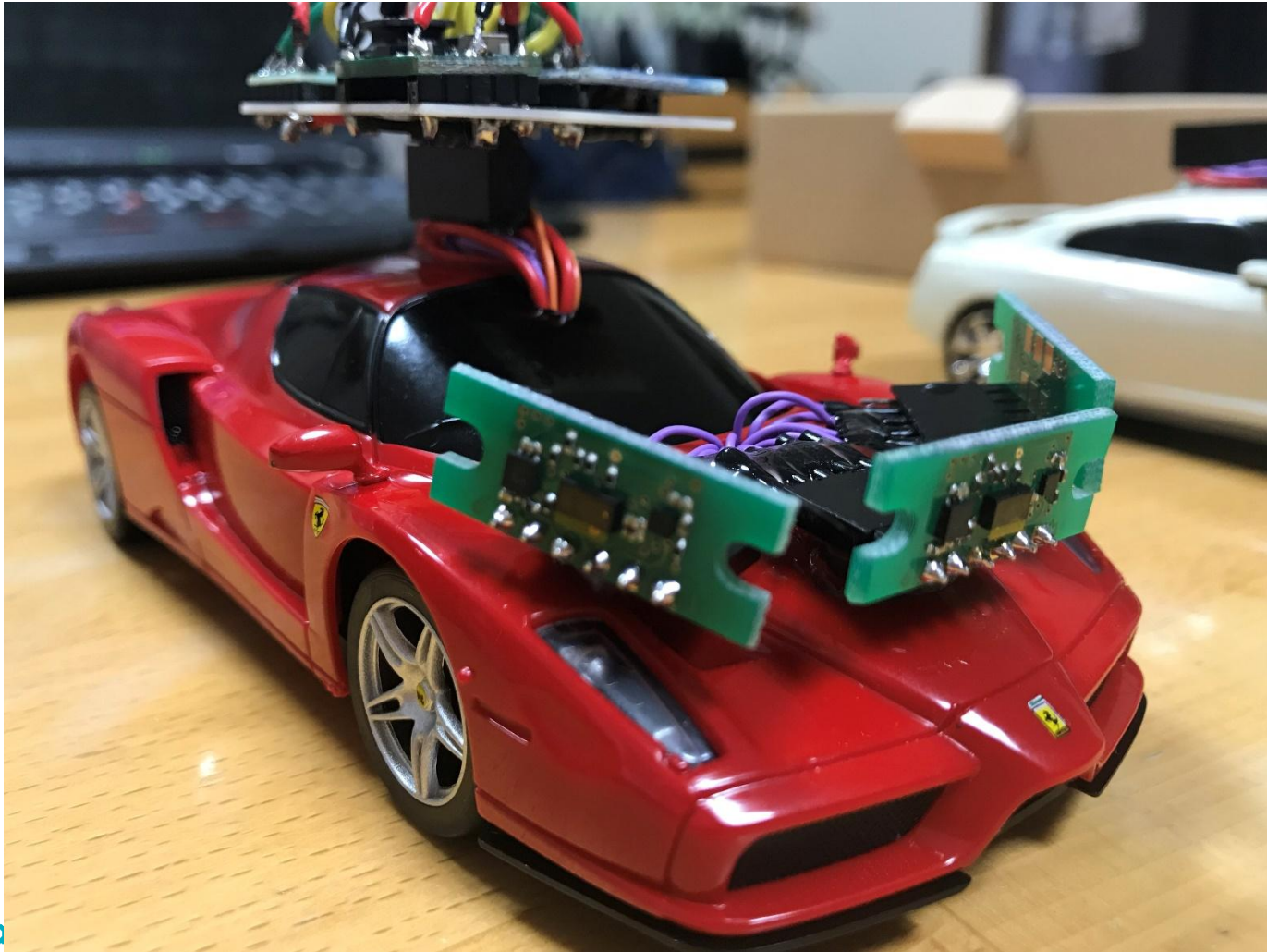


動作試験 (動画)

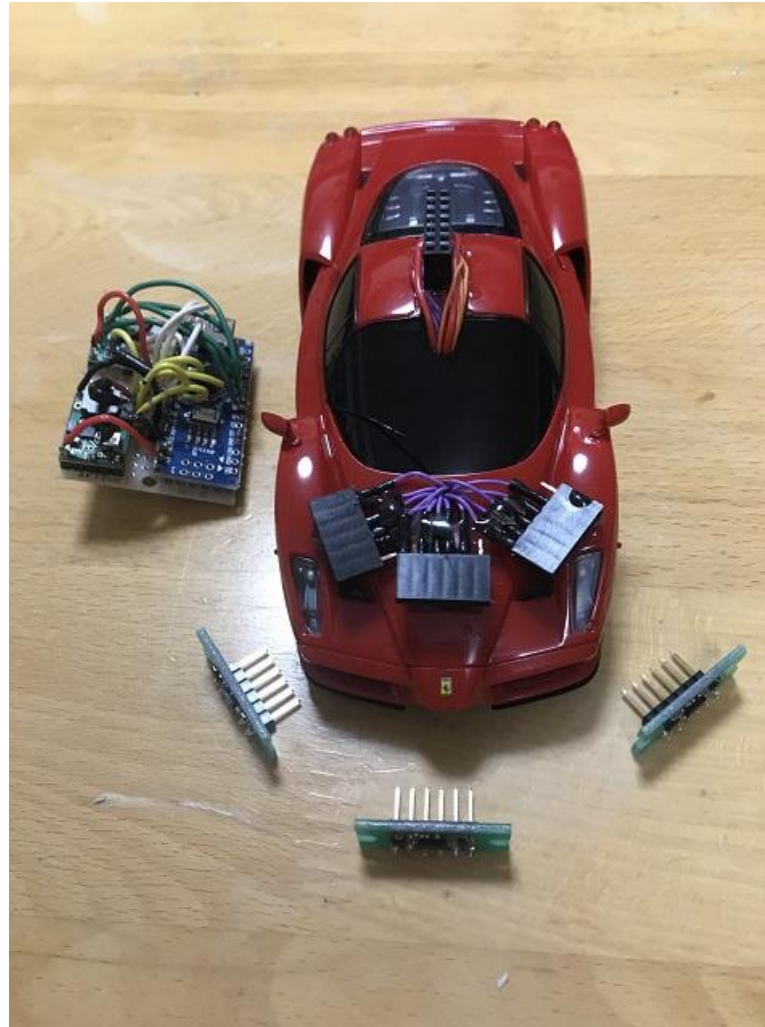
<https://youtu.be/dP815X6oKPw>



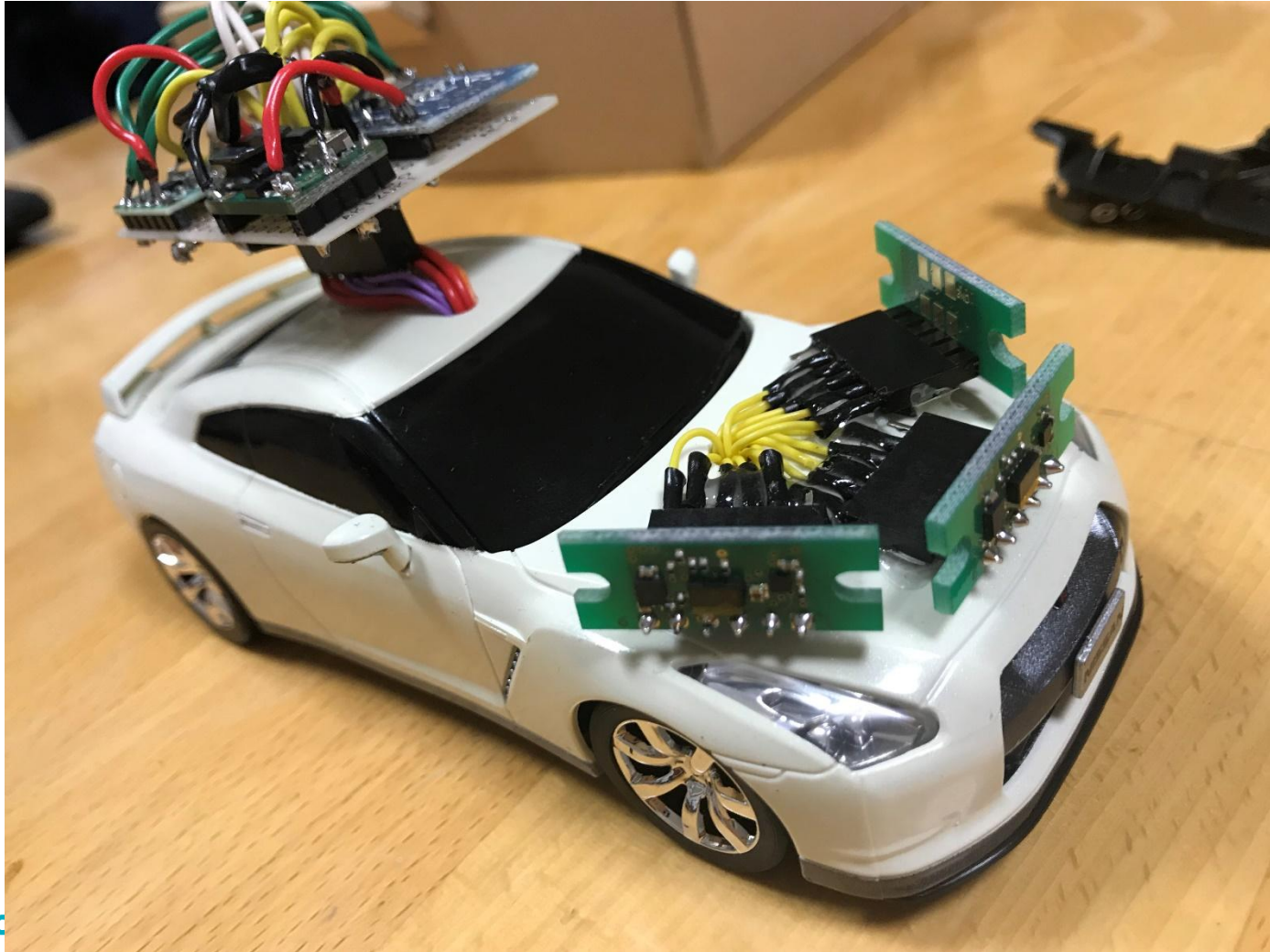
6号車



6号車



7号車



インターフェイスの標準化

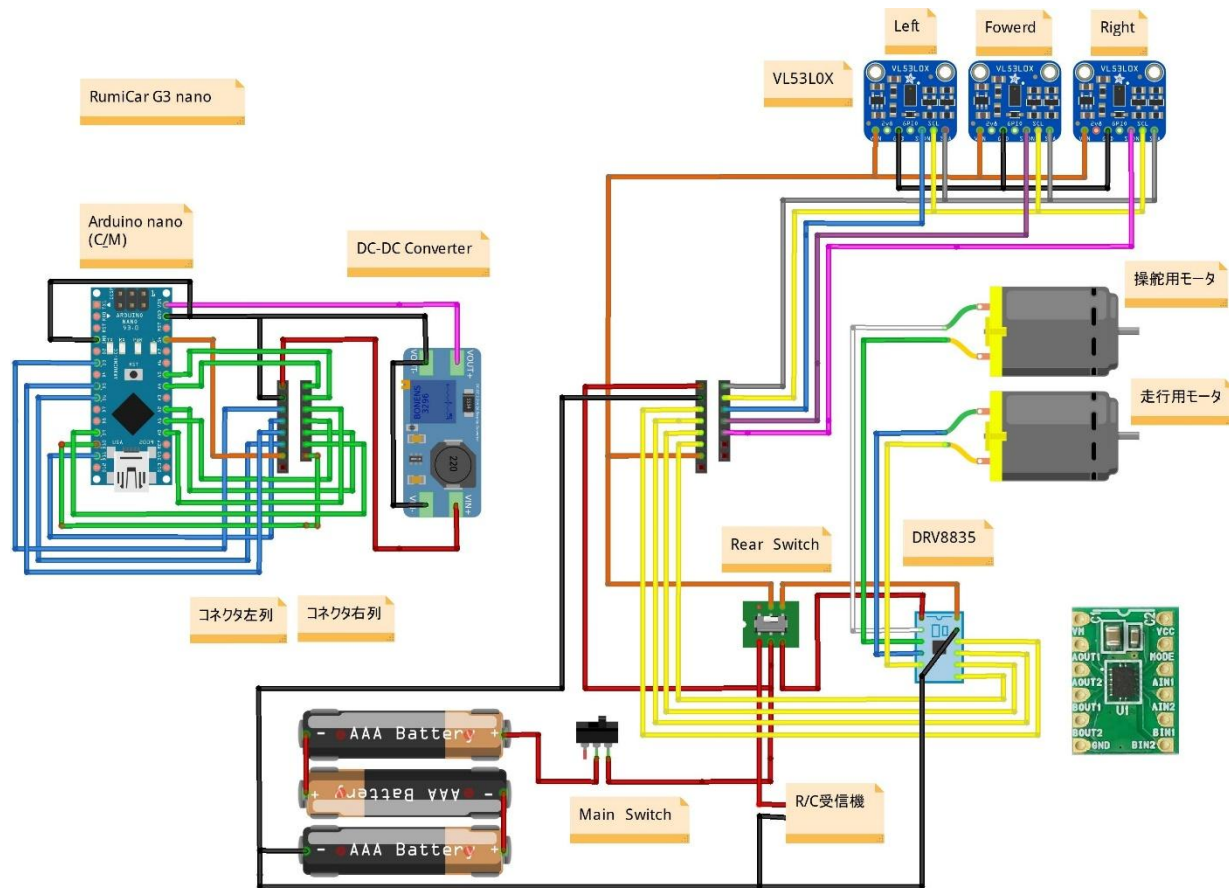


RumiCarの歴史と構成

号車	1号車	2号車	3号車	4号車	5号車
ベース車スケール	1/24		1/32		
コントローラー	Arduino UNO		Arduino Nano		モジュール式 (選択交換可能)
モータ制御	リレーシールド		MM-531	TA7291P	DRV8835
測距モジュール	GP2Y0A21YK		GP2Y0E02A		VL53L0X
測距方式	赤外線				レーザー
Arduino電源	単三乾電池 6 本 (9 V)		単四乾電池(4.5V)より LMR62421 を使い 7 V に昇圧		
測距モジュール 電源	Arduino より 供給		TA48033S で 3.3V を生成		

モータドライバBD6212HFPは小さすぎて使用断念

第3世代配線図

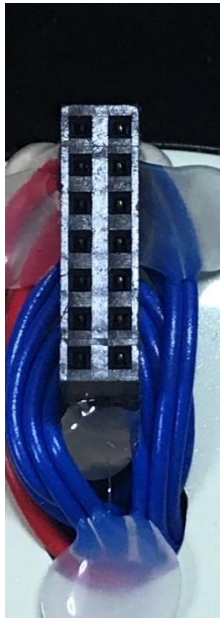


fritzing

インターフェイス

旧型式

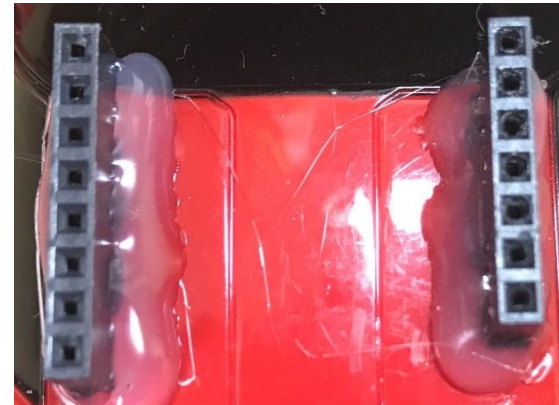
1
2
3
4
5
6
7



8
9
10
11
12
13
14

NEW

1
2
3
4
5
6
7
7.5



8
9
10
11
12
13
14

7.5 ← 誤挿入防止用ダミーピン

8×7ピン インターフェイス

表 3 第三世代車体-CM 接続インターフェイス(EX-IF)(旧表記)

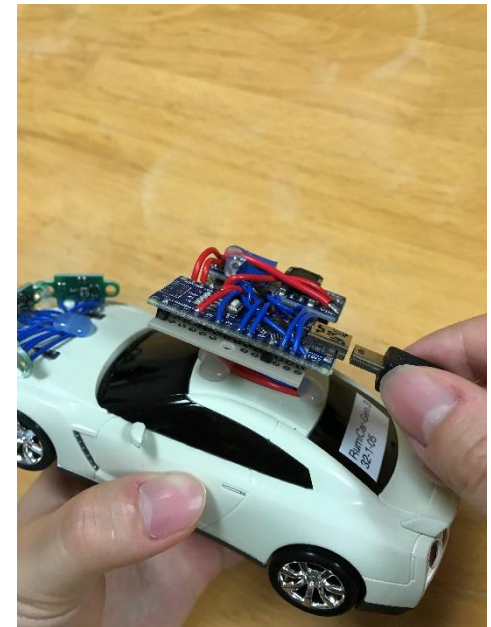
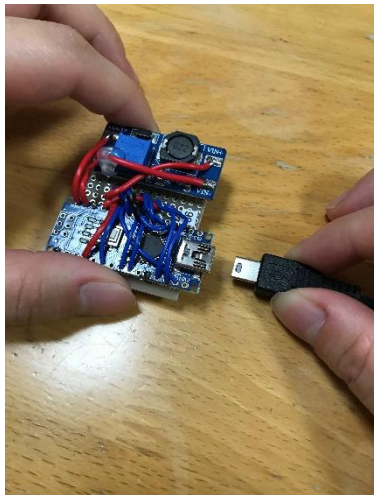
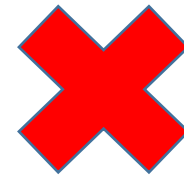
I/O	用途	ピン番号		用途	I/O
O	電源(4.5V)	1	8	SDA:E-GPIO21	I
O	GND	2	9	SCL:E-GPIO22	I
I	AIN1:右折側:A3:E-GPIO04	3	10	SHDN0(左):E-GPIO19	I
I	AIN2:左折側:A11:E-GPIO26	4	11	SHDN1(中):E-GPIO18	I
I	BIN1:前進側:A5:E-GPIO27	5	12	SHDN2(右):E-GPIO05	I
I	BIN2:後進側:A6:E-GPIO25	6	13	SERVO(操舵)用 A9 番ピン E-GPIO32	I
I	信号最大電圧及び電力供給(通常コンピュータの VCC)(5V または 3.3V)VL53L0X と DRV8835 の VCC へ	7	14	SERVO 用 A10 番ピン E-GPIO33	I
	誤挿入防止用のダミーピン (接続先は無し)	7.5			

誤挿入防止用7.5をダミーピンとして追加

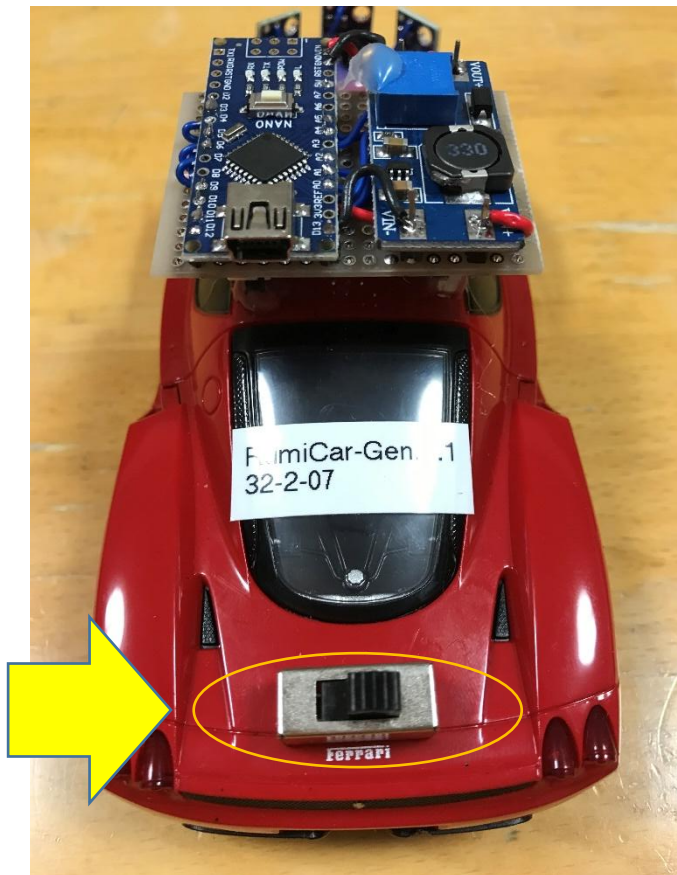
準備

注意点、必要なもの、準備

ケーブルの抜き差し注意点



コンピュータ制御へ設定



- (一部のモデルのみ)
- マニュアル制御とコンピュータ制御の切り替えスイッチ好きモデルは切り替えスイッチを右側へ切り替えます
- 右側がコンピュータ制御側です
- 右側にするとCM (ArduinoやESP32) からの命令で走ります
- (左側にするとラジコンで操作できるようになります)

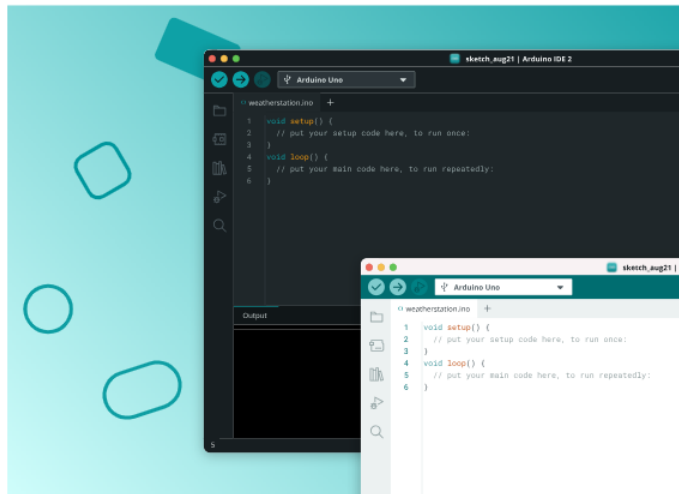
ケーブルの抜き差し注意点

- まず、コンピュータモジュールにUSBケーブルを接続する
- それから、コンピュータモジュール(以下CMと呼びます)を車体に接続する
- **注意！** 車体にCMを装着した状態でコンピュータモジュールにUSBケーブルを**接続しないこと**
- 車体に**CM**を装着したまま**USBケーブル**を抜かないこと
- CMコネクタ部分に過度な力が加わり破損の原因になります

Arduino IDEのインストール

- [URL:https://www.arduino.cc/en/Main/Software](https://www.arduino.cc/en/Main/Software)
- 「Windows Win 10 or newer(64-bit)」をインストール
- (既にインストール済の人はこの作業は飛ばしていただいて結構です)

Bring Your Projects to Life with Arduino Software



Arduino IDE 2.3.9

[Release notes](#)

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit)

DOWNLOAD



Nightly Builds

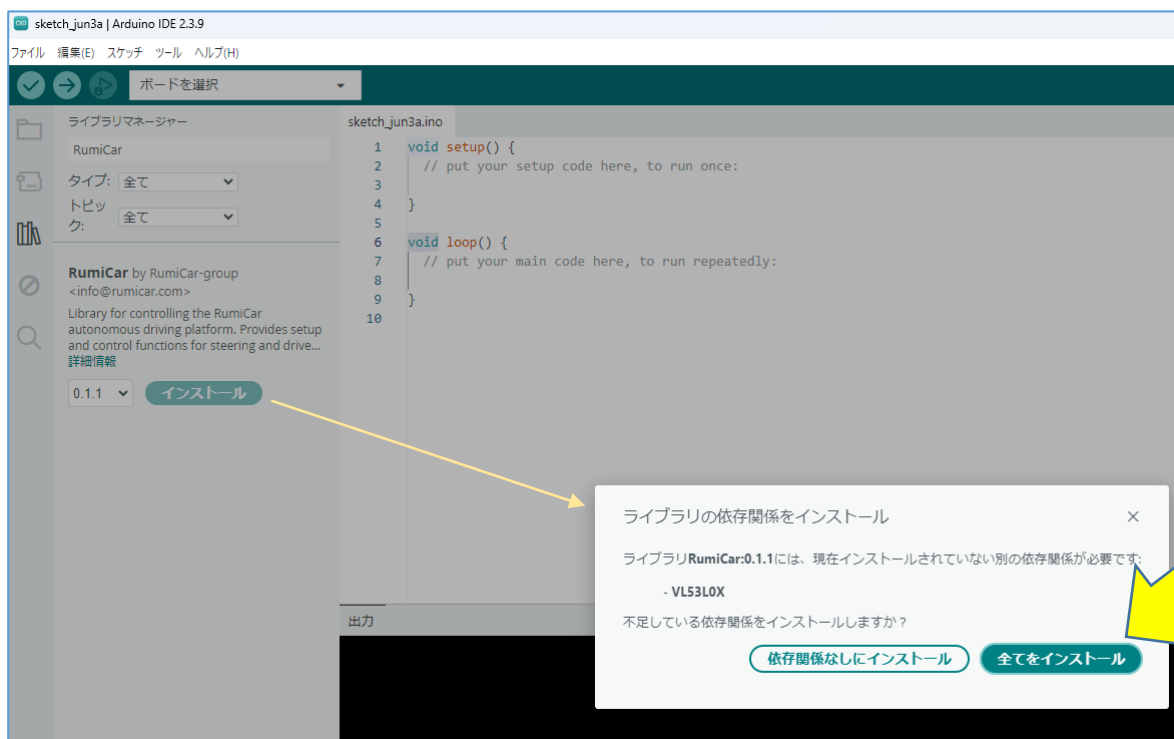
Download a preview of the incoming release with the most updated features and bugfixes.

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

RumiCarライブラリのインストール

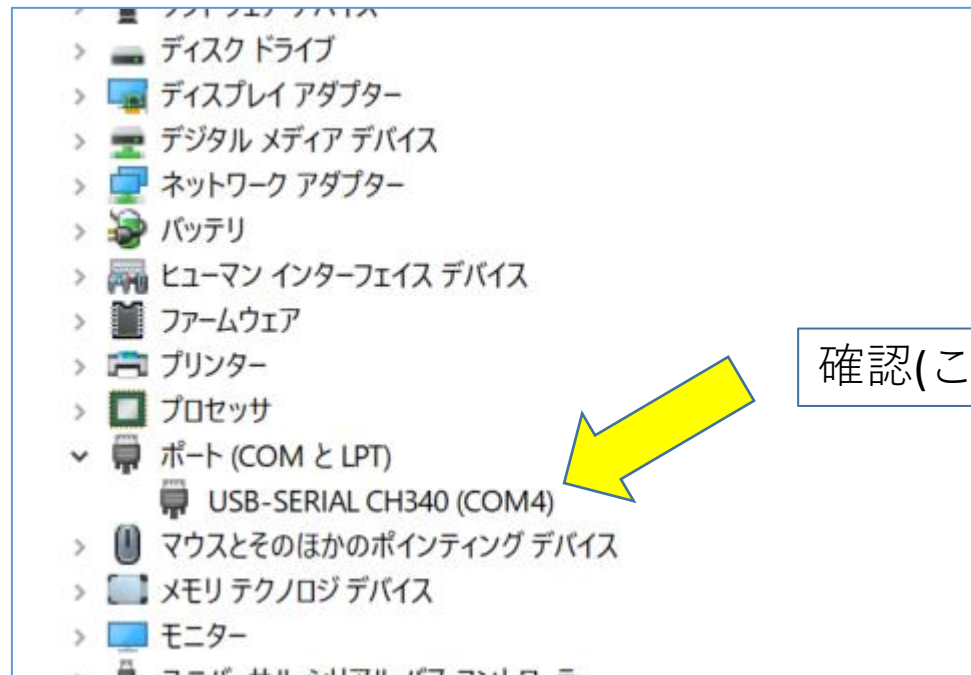
- Arduino IDEを開く

- 「ツール」->「ライブラリを管理...」で「RumiCar」を検索しインストールを押下する。依存関係も含めて「全てをインストール」する



COMポートの確認 (Windowsの場合)

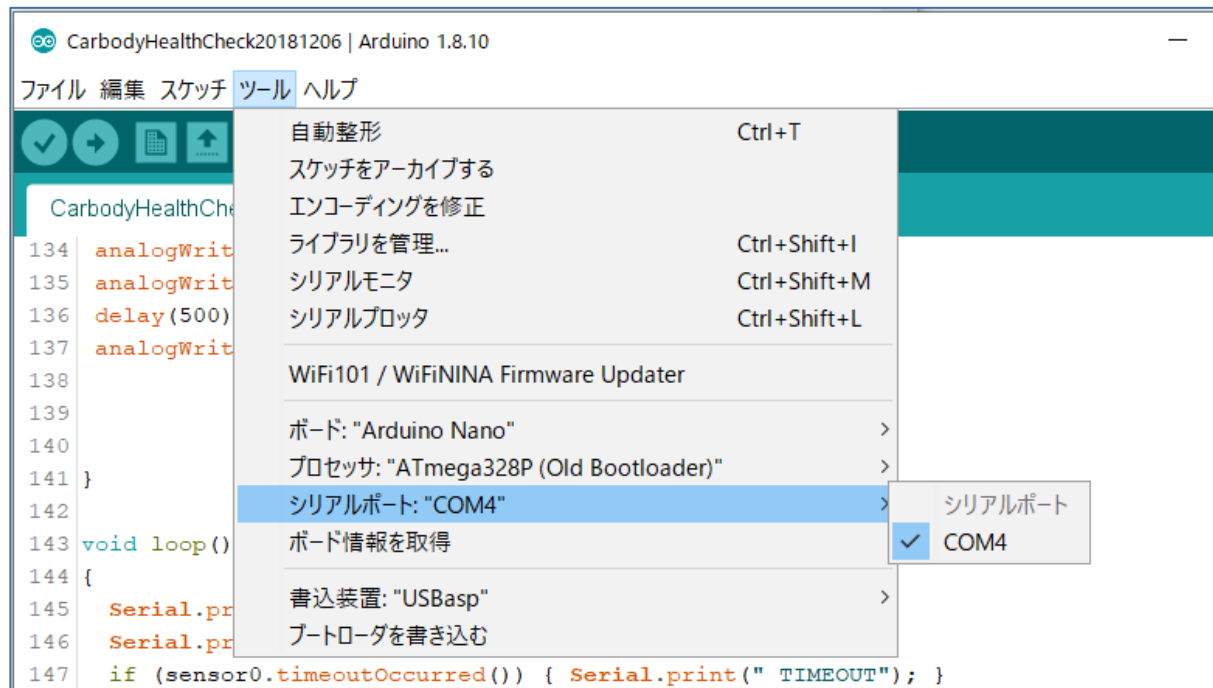
- RumiCarのコンピュータモジュールをUSBケーブルでパソコンに接続する
- デバイスマネージャで接続されたCOMポート番号を確認する。CH340を探します (COM番号を覚えておくこと)



確認(この例では4番)

Arduino IDEの設定

- ボード : Arduino Nano
- プロセッサ : ATmega328P(Old Bootloader)
- シリアルポート : デバイスマネージャで確認した番号



COMポートの確認 (Macの場合)

- Terminalソフトを起動します
- コマンド"`ls -l /dev/tty.usb*`"と入力することで接続されているCOMポートの一覧を表示させます
- Arduinoが接続されたCOMポートを確認する
- 下記の例では「`usbserial-14120`」(覚えておくこと)

```
mac2 — -bash — 80x24
Last login: Mon Aug 31 10:30:16 on ttys000

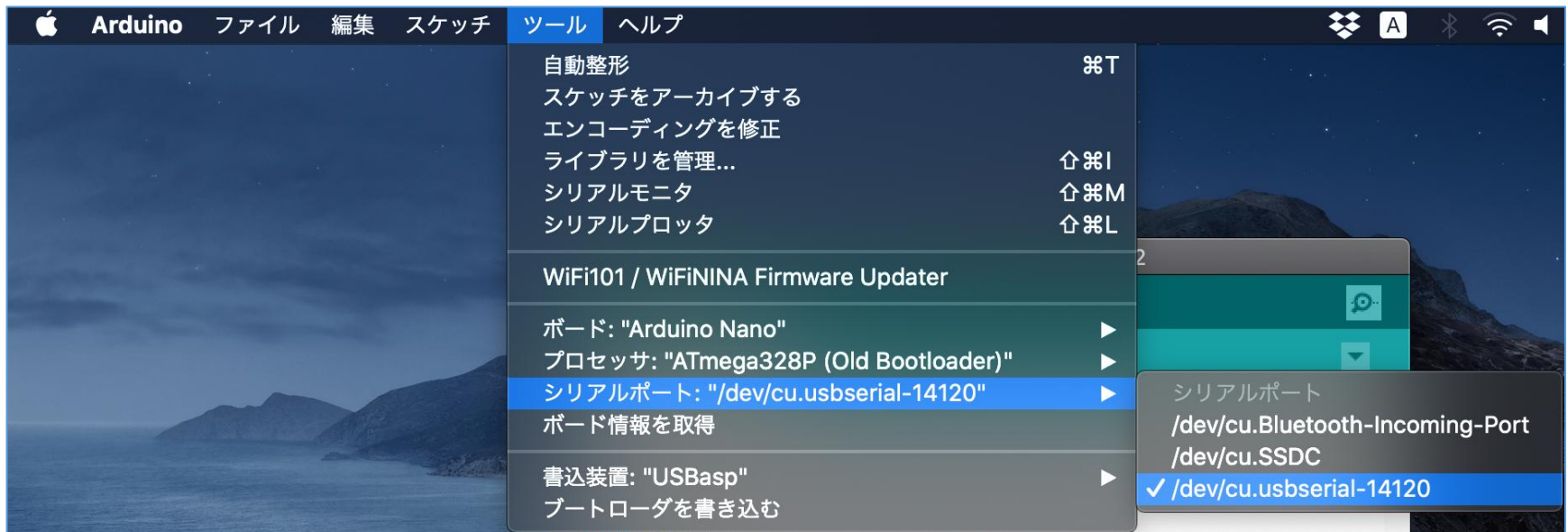
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
mac2noMacBook:~ mac2$ ls -l /dev/tty.usb*
crw-rw-rw- 1 root wheel 18, 4 8 31 10:31 /dev/tty.usbserial-14120
mac2noMacBook:~ mac2$
```

コマンド入力

結果表示

Arduino IDEの設定 (Macの場合)

- ボード : Arduino Nano
- プロセッサ : ATmega328P(Old Bootloader)
- シリアルポート : **Terminal**で確認したデバイス



設定の確認

- **重要！** もう一度、確認しよう！

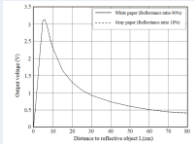
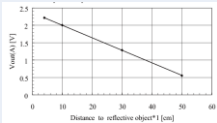
項目	設定	備考
ボード	Arduino Nano	Arduino UNO では動きません
プロセッサ	ATmega328P(Old Bootloader)	Old Bootloader のほう
シリアルポート	“デバイスマネージャで確認した番号”	ボードを変えると、変わったりしますので注意
書込装置	USBasp	ユーザ自身でデフォルトから変えている場合があります
シリアルポートのボーレート	9600bps	Arduino Nanoでは 9600bps に設定されている必要があります Arduino IDEのシリアルモニタを開いて確認します

距離の測定

レーザー測距モジュール

測距モジュール

• 測距モジュールの比較

型番	GP2Y0A21YK0F	GP2Y0E02A	VL53L0X	HC-SR04
測定方法	赤外線	赤外線	レーザー	超音波
出力	アナログ 	アナログ 	デジタル値	アナログ
推奨電源電圧	5V	3.3V	2.6-5.5V (ボード依存)	5V
消費電力	最大40mA	最大36mA	20mA	15mA
測定可能距離	80cm	50cm	2m	2m
RumiCarでの採用	1、2号車	3、4号車	5号車以降	非採用
備考	出力と距離が比例しない			高速移動時のエコーキャンセル処理問題

グラフはシャープ社データシートより

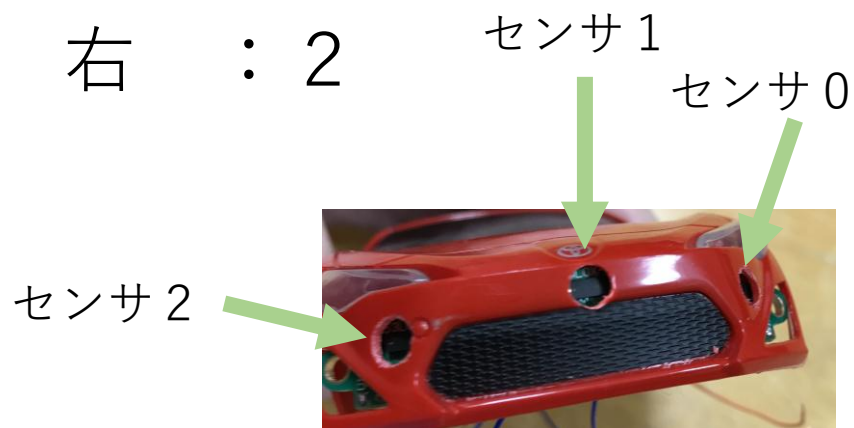
レーザー測距モジュール

- 車体前方に 3 基搭載

左 : 0

中央 : 1

右 : 2



VL53L0X

- RumiCarに搭載されているレーザー測距モジュールはVL53L0Xです
- VL53L0Xの特徴
 - 最大 2 m まで計測可能
 - I2Cで測定値を取得可能
 - I2Cアドレスはソフトウェアにより再設定可能
 - パラメータ設定により大まかにモード設定可能
 - 高速測定モード
 - 長距離測定モード
 - 高精度測定モード
 - サンプルプログラムは高速測定モードに設定されています

測距方法

- 各測距モジュールの測距値の取得は、例えば下記の命令を使います。結果は整数値（int）でmm（ミリメートル）の単位で得ることができます

`readRangeSingleMillimeters()`

- センサ1の測定値を取得するには例えば下記のようになります

```
i = sensor1.readRangeSingleMillimeters();
```

GitHubからExerciseをダウンロード

<https://github.com/RumiCar-group/RumiCar>

**【注意】 Exercise-A2.3と3.1は、
このイベントでは使用しません。**

algyan / RumiCar

Watch 1 Star 0 Fork 4

Code Issues 0 Pull requests 0 Actions Projects 0 Wiki Security 0 Insights Settings

「RumiCar/ハンズオン中継！自動運転アルゴリズムを楽しく手軽に体感しよう！#2」で使用するExerciseです。Exercise-A2.3とExercise-A3.1は上級編なので、イベント中には使わないです。 <https://algyan.connpass.com/event/169...>

Manage topics

15 commits 3 branches 0 packages 0 releases 3 contributors

Branch: master New pull request Create new file Upload files Find file Clone or download

Rumika685 Create README.md	Latest commit d4ba683 4 minutes ago
Exercise-1.1	Split to RumiCar Lib. 2 days ago
Exercise-1.2	Split to RumiCar Lib. 2 days ago
Exercise-2.1	Split to RumiCar Lib. 2 days ago
Exercise-2.2	Split to RumiCar Lib. 2 days ago
Exercise-2.3	Split to RumiCar Lib. 2 days ago
Exercise-2.4	Split to RumiCar Lib. 2 days ago
Exercise-3.1	Split to RumiCar Lib. 2 days ago
Exercise-3.2	Split to RumiCar Lib. 2 days ago
Exercise-A2.3	Split RumiCar Lib. 2 days ago
Exercise-A3.1	Split RumiCar Lib. 2 days ago
Exercise	Split to RumiCar Lib. 2 days ago
README.md	Create README.md 4 minutes ago



**Exerciseのヘッダー作製、
リファクタリング、上級編プログラムの追加を
してくださったのは
稲玉 繁樹 氏**

Exercise-1

距離を測ってみよう

Exercise-1.1

中央のセンサで測距しよう！

Exercise-1.1

中央のセンサで距離を測る(準備)

- RumiCarの車体の電源を**オフ**にします
- CMをRumiCarの車体より取り外してください
- CMにUSBケーブルを接続します
- USBケーブルをパソコンに接続します
- CMをRumiCarの車体に取り付けます

Exercise-1.1

中央のセンサで距離を測る

- Arduino IDEでファイル”Exercise”を開きます
- プログラムの下の方のところに計測値を表示するコードを追加します

```
225  
226 void loop()  
227 {  
228  
229 }
```



```
225  
226 void loop()  
227 {  
228   Serial.println(sensor1.readRangeSingleMillimeters());  
229 }
```

- Serial.print命令やSerial.println命令でシリアルモニタにデータを表示できるよ
- Serial.printは改行無し
- Serial.printlnは表示後改行ありだよ

Exercise-1.1

中央のセンサで距離を測る

- コンパイルとボードへの書き込み

押す



Exercise-1.1 | Arduino 1.8.13

ファイル 編集 スケッチ ツール ヘルプ

```
1 #include <RumiCar.h>
2
3 void setup()
4 {
5   RC_setup();
6 }
7 void loop()
8 {
9   Serial.println(sensor1.readRangeSingleMillimeters());
10 }
```

COM4のArduino Nano, ATmega328P (Old Bootloader)

Exercise-1.1

中央のセンサで距離を測る（続き）

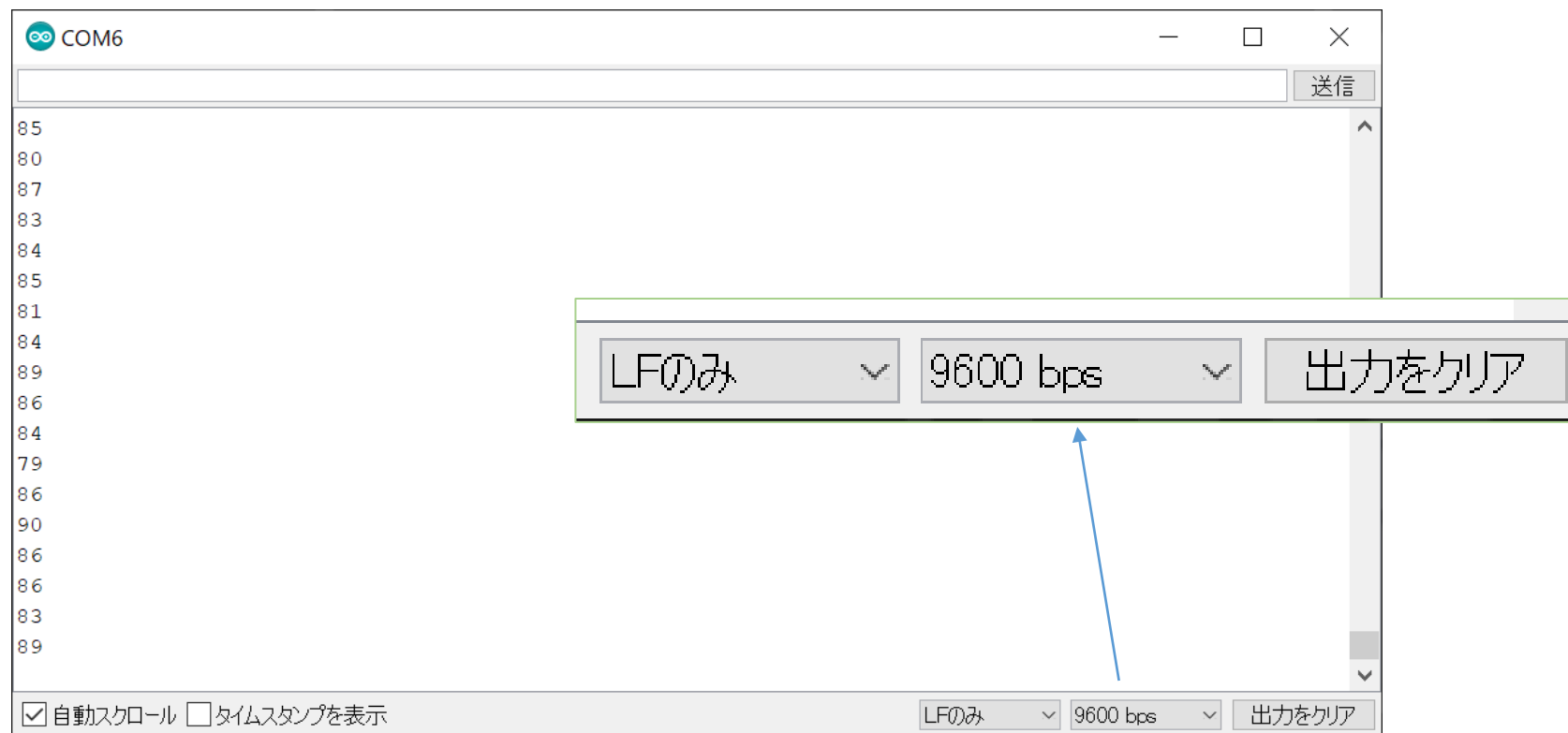
- シリアルモニタで測定値を表示させます
 - ツール -> シリアルモニタ



Exercise-1.1

中央のセンサで距離を測る（続き）

- うまく表示できない場合はボーレートを確認してみてください。
(9600 bpsになっている必要があります)



Exercise-1.1

中央のセンサで距離を測る（続き）

- うまく表示できない場合は「準備」の項の「設定の確認」のページの内容を再度確認してください

Exercise-1.2

3つのセンサで測距だ！

Exercise-1.2

3つのセンサで距離を測る

- ファイル”Exercise-1.2”を開きます

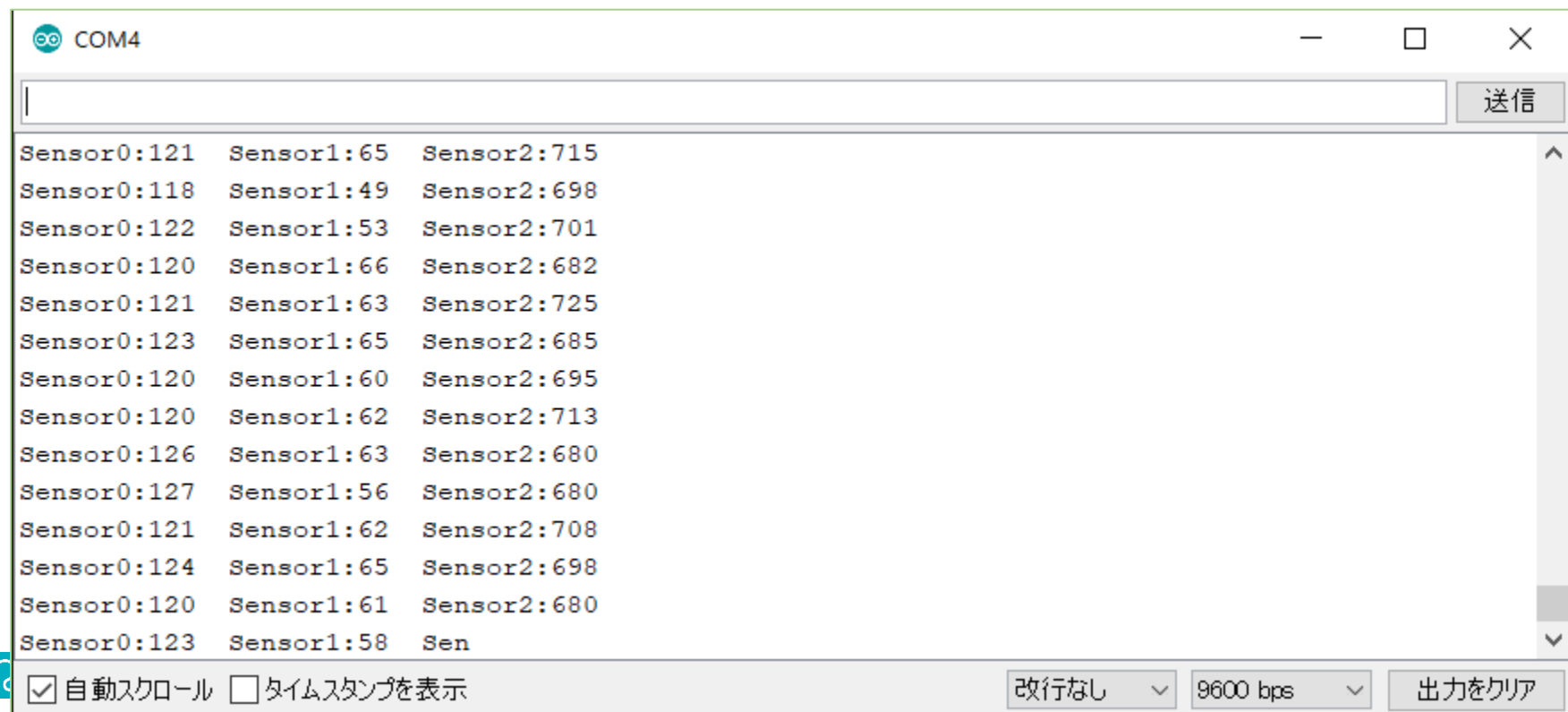
```
123
124 void loop()
125 {
126     Serial.print("Sensor0:");
127     Serial.print(sensor0.readRangeSingleMillimeters());
128     Serial.print("  Sensor1:");
129     Serial.print(sensor1.readRangeSingleMillimeters());
130     Serial.print("  Sensor2:");
131     Serial.println(sensor2.readRangeSingleMillimeters());
132 }
```

センサ	センサの場所
Sensor0	左
Sensor1	中央
Sensor2	右

Exercise-1.2

3つのセンサで距離を測る（続き）

- シリアルモニタで測定値を表示させる
 - ツール -> シリアルモニタ



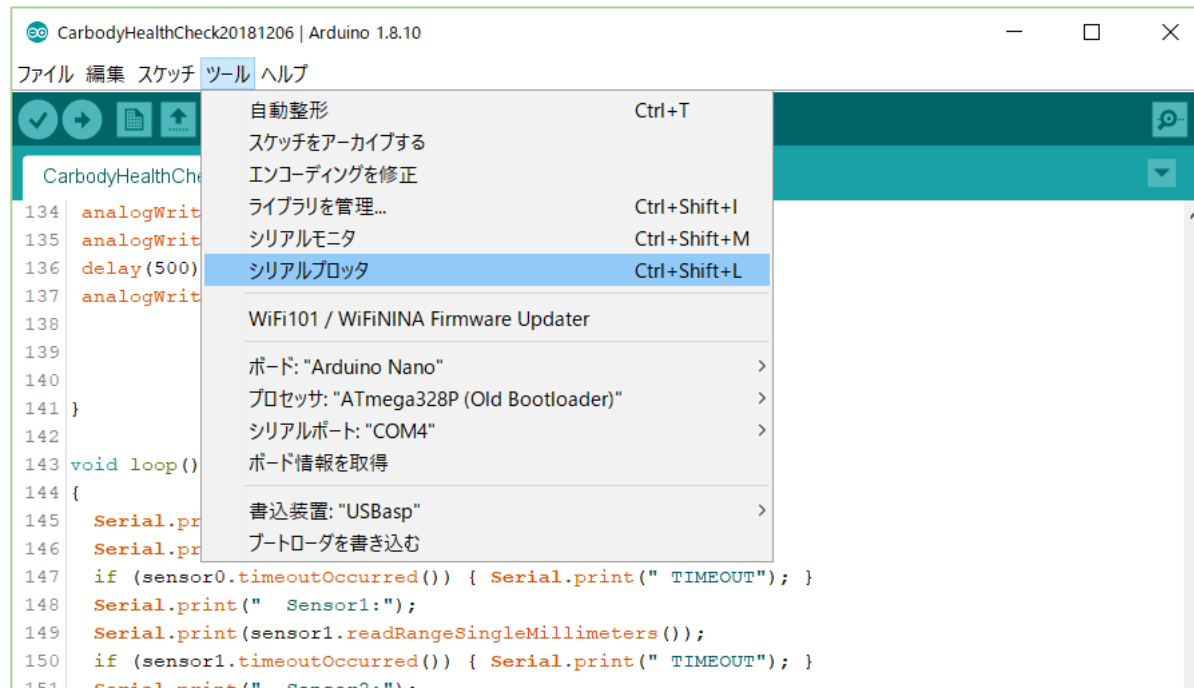
Exercise-1.3

シリアルプロッタを使ってみよう

Exercise-1.3

シリアルプロッタを使ってみよう

- (プログラムはそのままExercise-1.2のものを使います)
- シリアルモニタを閉じる
- ツール -> シリアルプロッタ
 - ※シリアルモニタのウィンドウが開いている場合は閉じる必要があります



休憩室



	紫外線硬化樹脂	ホットメルト接着剤 (通称グルー)
長所	• 短時間で固まる • 強度も高い • 透明で見栄えが良い • 重ね塗りが可能	• 安い • 失敗しても剥がしてやり直しが効く
短所	• 長期間経過すると脆くなり崩れる	• 透明でないの見栄えが良くない (イ〇スタ映えしない) • 紫外線硬化樹脂程強度がない



モータ制御

ハンドルを動かす、走る

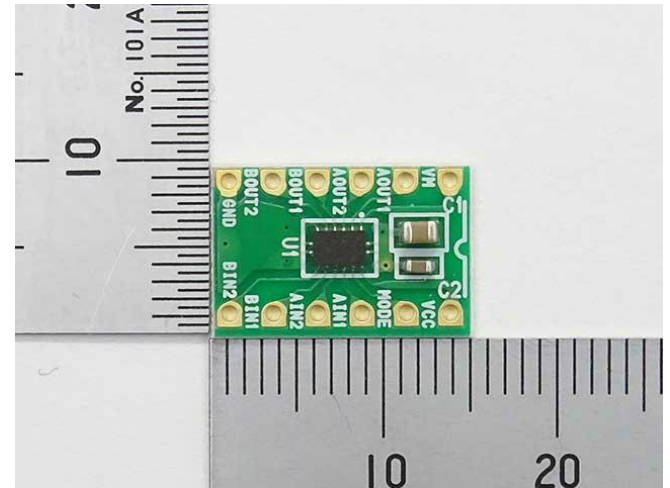
モータドライバ

- RumiCarに搭載されているモータドライバは DRV8835 です

- DRV8835の特徴

[DRV8835](#)

- 2チャンネル
- 各チャンネル1.5Aまで流せる
- PWM(250kHz)
- 低消費電力
- 低電圧降下（モータ側）
- ロジック電源（VCC）とモータ電源が独立している
 - ロジック電源電圧は2から7V
 - モータ電源電圧は11Vまで



DRV8835諸元(1/2)

$T_A = 25^\circ\text{C}$ (unless otherwise noted)

		MIN	NOM	MAX	UNIT
V_{CC}	Device power supply voltage	2		7	V
V_M	Motor power supply voltage	0		11	V
V_{IN}	Logic level input voltage	0		V_{CC}	V
I_{OUT}	H-bridge output current ⁽¹⁾	0		1.5	A
f_{PWM}	Externally applied PWM frequency	0		250	kHz

(1) Power dissipation and thermal limits must be observed.

DRV8835 Dual Low-Voltage H-Bridge ICデータシート 2016年8月9日 より

DRV8835諸元(2/2)

$T_A = 25^\circ\text{C}$, $V_M = 5\text{ V}$, $V_{CC} = 3\text{ V}$ (unless otherwise noted)

PARAMETER		TEST CONDITIONS	MIN	TYP	MAX	UNIT
POWER SUPPLY						
I _{VM}	VM operating supply current	No PWM, no load		85	200	μA
		50 kHz PWM, no load		650	2000	
I _{VMQ}	VM sleep mode supply current	V _M = 2 V, V _{CC} = 0 V, all inputs 0 V		5		nA
		V _M = 5 V, V _{CC} = 0 V, all inputs 0 V		10	95	
I _{VCC}	VCC operating supply current			450	2000	μA
V _{UVLO}	VCC undervoltage lockout voltage	V _{CC} rising			2	V
		V _{CC} falling			1.9	
LOGIC-LEVEL INPUTS						
V _{IL}	Input low voltage			0.3 × V _{CC}		V
V _{IH}	Input high voltage		0.5 × V _{CC}			V
I _{IL}	Input low current	V _{IN} = 0	−5		5	μA
I _{IH}	Input high current	V _{IN} = 3.3 V			50	μA
R _{PD}	Pulldown resistance		100			kΩ
H-BRIDGE FETS						
R _{DS(ON)}	HS + LS FET on resistance	V _{CC} = 3 V, V _M = 3 V, I _O = 800 mA, T _J = 25°C		370	420	mΩ
		V _{CC} = 5 V, V _M = 5 V, I _O = 800 mA, T _J = 25°C		305	355	
I _{OFF}	OFF-state leakage current				±200	nA
PROTECTION CIRCUITS						
I _{OCP}	Overcurrent protection trip level		1.6		3.5	A
t _{DEG}	Overcurrent de-glitch time		1			μs
t _{OCR}	Overcurrent protection retry time		1			ms
t _{DEAD}	Output dead time		100			ns
t _{TSD}	Thermal shutdown temperature	Die temperature	150	160	180	°C

IN/INモード

- IN/INモードとPHASE/ENABLEモードを持っています
- RumiCarではDRV8835をIN/INモードで利用します

Table 3. IN/IN Mode

MODE	xIN1	xIN2	xOUT1	xOUT2	FUNCTION (DC MOTOR)
0	0	0	Z	Z	Coast
0	0	1	L	H	Reverse
0	1	0	H	L	Forward
0	1	1	L	L	Brake

DRV8835 Dual Low-Voltage H-Bridge ICデータシート 2016年8月9日 より

Phase/Enableモードの場合

Table 4. Phase/Enable Mode

MODE	xENABLE	xPHASE	xOUT1	xOUT2	FUNCTION (DC MOTOR)
1	0	X	L	L	Brake
1	1	1	L	H	Reverse
1	1	0	H	L	Forward

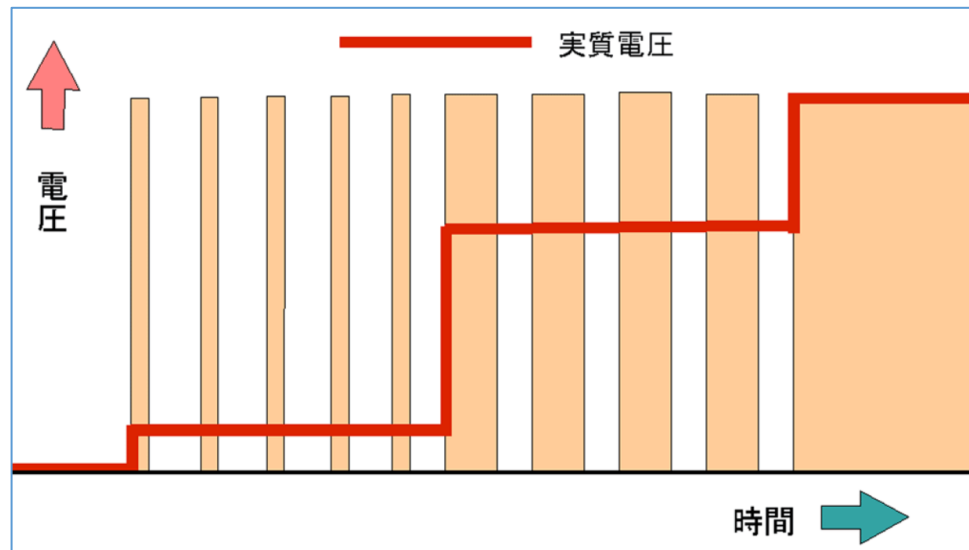
- Enableが1の時、モーターが回転
- Enableが0の時、モーターが停止
- Phaseが1の時は逆回転の向き
- Phaseが0の時は正転の向き

Enableが回転数、Phaseが回転方向

DRV8835 Dual Low-Voltage H-Bridge ICデータシート 2016年8月9日 より

PWMの説明

- RumiCarでは走行速度制御にPWM技術を利用しています
- PWMとは”Pulse Width Modulation”の略語で電圧を短い間隔でオンとオフを繰り返すことによって見かけの電圧を変化させる技術です



ArduinoのPWM

- ArduinoのPWMはタイマと連動しています
- Arduinoのタイマは3個あります

タイマ	ピン	周波数
タイマ0	5、6	980Hz
タイマ1	9、10	490Hz
タイマ2	3、11	490Hz

- ArduinoのServoライブラリを使うとタイマ1は使えなくなってしまうです

Arudino版でのピンアサイン		
機能	タイマ	ピン
操舵	タイマ2	3、11
走行	タイマ0	5、6

RumiCarでのPWM制御

- RumiCarのプログラムサンプルではピンにマクロ設定をしています
- 例えば右にハンドルを曲げたいときは**AIN1**をオン、**AIN2**を”0”に設定します
- ※この例は**Arduino**版のピンアサインです。**ESP**版などはピンアサインが異なりますがマクロを使ってピンアサインの違いを意識することなくプログラムを開発できるようにしてあります

タイマ	機能	ピン	マクロ設定
タイマ2	右折	3	AIN1
	左折	11	AIN2
タイマ0	前進	5	BIN1
	後進	6	BIN2

RumiCarでのRC関数

- RumiCarではプログラムを開発しやすいように下記関数を用意しました

動作	関数の型	関数名	第一引数 (int)	第二引数 (int)	動作	エラー時の戻り値
操舵	int	RC_steer	RIGHT	(無し)	右操舵	0
			LEFT		左操舵	
			CENTER		中央	
走行	int	RC_drive	FREE	0から255 (PWM値)	ニュートラル	0
			REVERSE		後進	
			FORWARD		前進	
			BRAKE		ブレーキ	

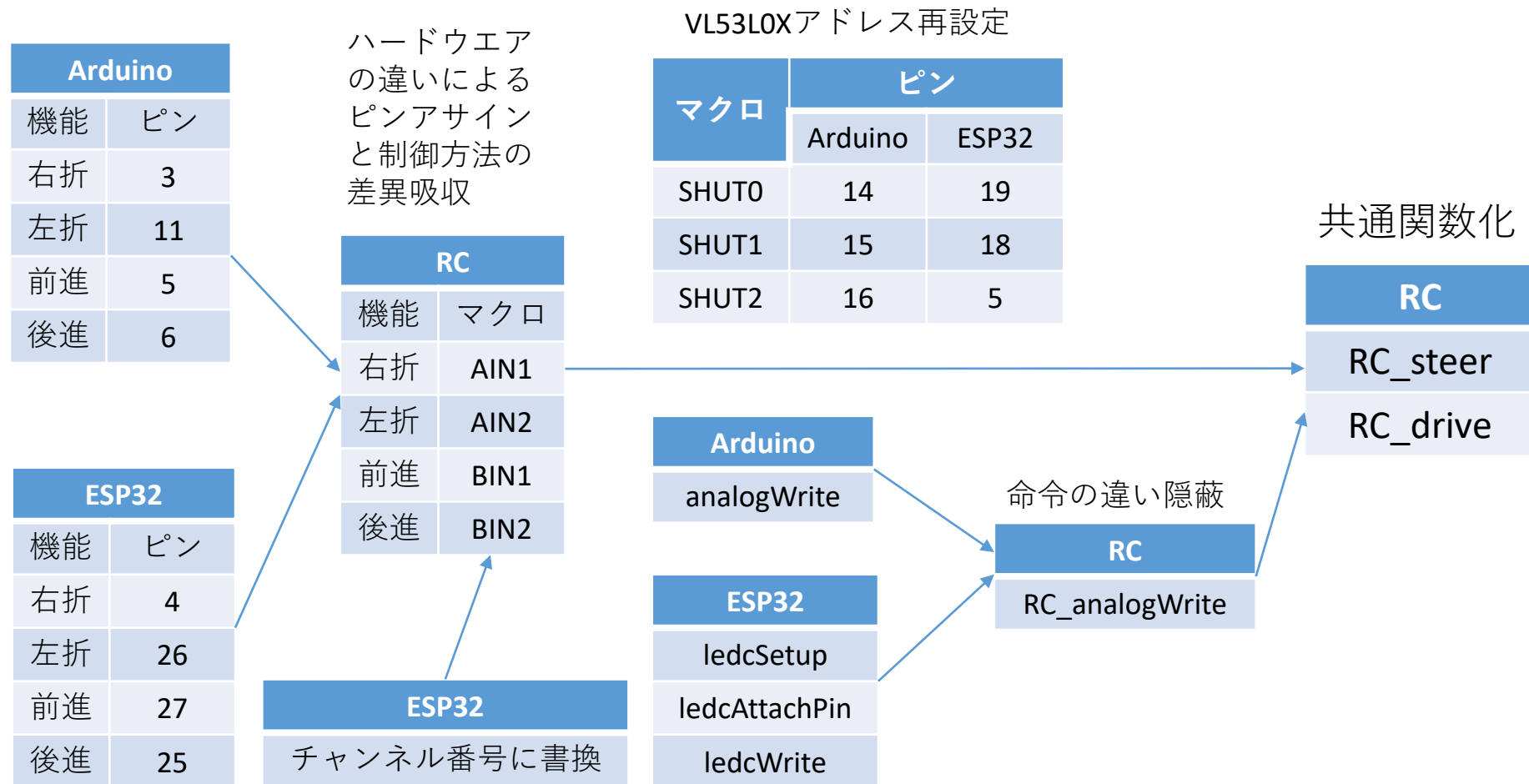
- RC_driveのFREE時の第二引数の値は指定は必要ですが値自体は無視されます
- RC_driveの第二引数の値はPWM値であり値が大きいほどモータに電力を掛けることができます
- 第一引数はint型です。この表の値を指定すると最終的にマクロで展開されます



関数を用意することの意義

- RumiCarではマクロ設定や独自関数としてRC関数を追加するなどにより下記のメリットを提供します
 - 複数の命令をまとめているのでプログラムがすっきりし可読性が向上する
 - 固定の引数の記載を省くことによりプログラムがすっきりし可読性が向上する
 - 不要な記載を省くことによるプログラムミス低減
 - ハードウェアの差異によるピンアサインや制御方法等の考慮の不要化
 - コンピュータモジュール間の差異を吸収しプログラムの共通化

共通化ストラクチャ



RC_steer関数の利用例

- RC_steer関数を利用した操舵プログラミングの例です
- 左側にオリジナル命令での記述を示します

オリジナル命令での記述

```
123  
124 void loop()  
125 {  
126   analogWrite(AIN1,255);  
127   analogWrite(AIN2,0);  
128   delay(500);  
129   analogWrite(AIN1,0);  
130   analogWrite(AIN2,255);  
131   delay(500);  
132 }
```



RC関数利用の記述

```
210  
211 void loop()  
212 {  
213   RC_steer(RIGHT);  
214   delay(500);  
215   RC_steer(LEFT);  
216   delay(500);  
217 }
```

Exercise-2

モータ制御

Exercise-2.1

ハンドルを切る

Exercise-2.1

ハンドルを切る(準備)

- RumiCarの車体の電源を**オフ**にします
- CMをRumiCarの車体より取り外してください
- CMにUSBケーブルを接続します
- USBケーブルをパソコンに接続します
- ファイル”Exercise-2.1”を開きます

Exercise-2.1

ハンドルを切る（プログラム）

```
210
211 void loop()
212 {
213     RC_steer(RIGHT);
214     delay(500);
215     RC_steer(LEFT);
216     delay(500);
217 }
```

- **delay**命令はそのまま何もしないで指定時間だけ待つよ
- **delay**の引数の単位はミリ秒だよ
- このプログラムはハンドルを切ったまま**0.5秒(500ミリ秒)**そのままハンドルを保持するよ

- ハンドルを右に切る
- 0.5秒そのまま
- ハンドルを左に切る
- 0.5秒そのまま

繰り返す

Exercise-2.1

ハンドルを切る(試験)

- コンパイルしてプログラムをCMに書き込みます
- CMからUSBケーブルを取り外します
- CMをRumiCarの車体に取り付けます
- RumiCar車体の電源をオンにしてください
- ハンドルが左右に曲がっていますか？

Exercise-2.2

速度制御

Exercise-2.2 速度制御(準備)

- RumiCarの車体の電源を**オフ**にします
- CMをRumiCarの車体より取り外してください
- CMにUSBケーブルを接続します
- USBケーブルをパソコンに接続します
- ファイル”Exercise-2.2”を開きます

Exercise-2.2

速度制御(プログラム)

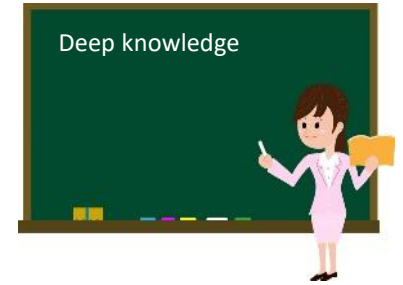
```
210
211 void loop()
212 {
213     RC_drive(FORWARD, 255);
214     delay(500);
215     RC_drive(FORWARD, 200);
216     delay(500);
217     RC_drive(FORWARD, 150);
218     delay(500);
219 }
```

- RC_driveの第二引数、255とか200とか150はPWMの値です
- 255が最大で128が最大値の約半分の値になります
- 値を下げてゆくとタイヤの回転もだんだんと遅くなります

Exercise-2.2

速度制御(実走試験)

- CMからUSBケーブルを取り外します
 - CMをRumiCarの車体に取り付けます
 - まず、いきなり走らせないでRumiCarの車体を手で持ち上げたままにします
 - RumiCar車体の電源をオンにしてください
 - タイヤの回転が変化していますか？
-
- RumiCarをそっと机に置いてみましょう



Deep knowledge

直流モータの低回転制御

稲玉 繁樹 氏

RumiCarのモータ制御

Deep knowledge



- RumiCarは2つのモータを制御しています。

- ステアリング

```
//操舵の関数
```

```
int RC_steer (int direc ) {
```

```
    RC_analogWrite(AIN1,255);  
    RC_analogWrite(AIN2,0);
```

オンオフ制御

全力でモータをオンしている

- ・角度が決められない
- ・電流が流れすぎている

- 走行

```
//走行の関数
```

```
int RC_drive(int direc, int ipwm) {
```

```
    RC_analogWrite(BIN1,0);  
    RC_analogWrite(BIN2,ipwm);
```

PWM制御

電圧を変えている

- 車両条件などで速度安定しない
- 速度と電圧が比例しない

RumiCarのモータ制御

Deep knowledge



- 低速で動かしたいが，走行指令 120/255 以下は動かない

- 原因 1 : PWM周期が高すぎる
- 原因 2 : DCモータは始動時に大きい電流が必要



$$Z = j\omega L \cdots (1) \quad \omega = 2\pi f \cdots (2)$$

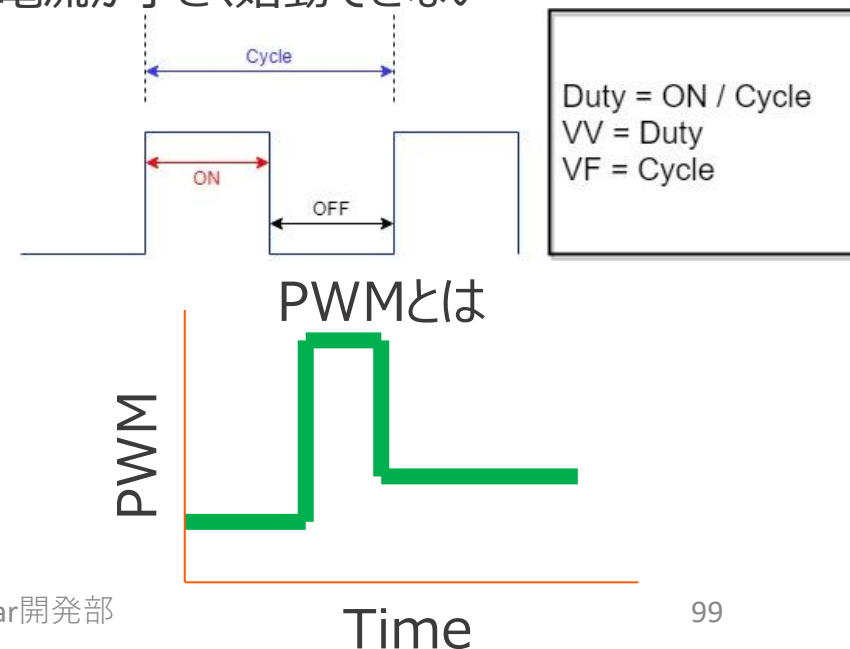
≡ 見かけの抵抗が高くなりPWM指令が低いと電流が小さく始動できない
よって，使えるPWM範囲が狭くなっている。

- 対策 1 : PWM周期を変更

- 976kHz → 244Hz

- 対策 2 : 始動ブーストを導入

- 始動時の数10ms はPWM大きく



Advanced ソフトの解説


 Deep knowledge

□ A2-3 低速で走らせるため

```
// TCCR0B |= B00000011;           // 64分周= 16M/ 64/256=976Hz #default
TCCR0B |= B00000100;           // 256分周= 16M/ 256/256=244Hz
```

```
// 始動後に低速へ
```

```
RC_drive(FORWARD, 200);    RC_delay(20);
RC_drive(FORWARD, 64);     RC_delay(500);
```

PWM周期を下げる

→ 低い値でも電流が流れる
始動時強く、動き出したら弱く
→ 40位まで動くようになりました

□ B2-3 低速動作をライブラリへ

```
// Private Variable
#define UP_SPEED      (200)      // StartUP speed
#define UP_TIME       (3)       // 30[ms]
```

```
RC_drive(FORWARD, 40);    RC_delay(1000);
RC_drive(BRAKE, 255);     RC_delay(500);
RC_drive(REVERSE, 40);    RC_delay(1000);
RC_drive(BRAKE, 255);     RC_delay(500);
```

PWMドライバを定時割込化

始動時ブーストを自動化
→ 意識せずに低速動作が可能

□ B2-5 縦列駐車にチャレンジ

動画で確認！

注：PWM周期変更によりDelay()の周期がずれるため、RC_delay()を使うこと

Exercise-2.3

前進と後進

Exercise-2.3

前進と後進(準備)

- RumiCarの車体の電源を**オフ**にします
- CMをRumiCarの車体より取り外してください
- CMにUSBケーブルを接続します
- USBケーブルをパソコンに接続します
- ファイル”Exercise-2.3”を開きます

Exercise-2.3

前進と後進(プログラム)

- RC_driveの第一引数がFORWARDを指定した時に前進
- 第一引数がREVERSEの時に後進

```
210
211 void loop()
212 {
213     RC_drive(FORWARD, 255);
214     delay(500);
215     RC_drive(REVERSE, 255);
216     delay(500);
217 }
```

- 前進する
- 0.5秒そのまま（前進し続ける）
- 後進する
- 0.5秒そのまま（後進し続ける）

繰り返す

Exercise-2.3

前進と後進(実走試験)

- コンパイルしプログラムをCMへ書きこみます
- CMからUSBケーブルを取り外します
- CMをRumiCarの車体に取り付けます
- まず、いきなり走らせないでRumiCarの車体を手で持ち上げたままにします
- RumiCar車体の電源をオンにしてください
- タイヤが前進と後進を繰り返してますか？
- RumiCarをそっと机に置いてみましょう

Exercise-2.3

前進と後進(発展問題)

- (発展問題) なるべくスリップしないようにして前進と後進を繰り返すようにするにはどうしたら良いでしょうか？

Exercise-2.4

ジグザグ走行

Exercise-2.4

ジグザグ走行（準備）

- RumiCarの車体の電源を**オフ**にします
- CMをRumiCarの車体より取り外してください
- CMにUSBケーブルを接続します
- USBケーブルをパソコンに接続します
- ファイル”Exercise-2.4”を開きます

Exercise-2.4

ジグザグ走行(プログラム)

```
210
211 void loop()
212 {
213   RC_drive(FORWARD, 255);
214   RC_steer(RIGHT);
215   delay(500);
216   RC_steer(LEFT);
217   delay(500);
218 }
```

- 前進する
- ハンドルを右に切る
- 0.5秒そのまま（右に進み続ける）
- ハンドルを左にきる
- 0.5秒そのまま（左に進み続ける）

繰り返す

Exercise-2.4

ジグザグ走行(実走試験)

- コンパイルしプログラムをCMへ書きこみます
- CMからUSBケーブルを取り外します
- CMをRumiCarの車体に取り付けます
- まず、いきなり走らせないでRumiCarの車体を手で持ち上げたままにします
- RumiCar車体の電源をオンにしてください
- RumiCarは予想通りの動きをしていますか？
- RumiCarをそっと机に置いてみましょう

自律走行基礎

センサによる車体制御

自律走行基礎

- 自律走行はセンサ等で得られた情報や、既に自分が持っている情報など、またそれらを組み合わせて自動車を自動で最適に運転させることです
- この章では自律走行の基礎として簡単な例を考えます

自律走行基礎(状況の仮定)

- 次のような単純な自律走行の状況を仮定します
 - 前進走行中に前方に障害物を検知したら障害物に衝突しないように安全に停車する
 - 障害物を検知して停車中に障害物がなくなったら再度前進を再開する

Exercise-3

自律走行プログラムを開発しよう！

Exercise-3.1

安全に停止する車

Exercise-3.1

安全に停止する車

- 単純な条件として具体的に下記を仮定します
 - RumiCarの中央センサ(Sensor1)で障害物を検知しながら、障害物が検知されない場合は前進します
 - 障害物の例えば10cm手前で停車、障害物がなくなったら再度前進再開します

Exercise-3.1

RumiCarでの開発にあたっての考慮事項

- 距離の測定誤差
 - サンプルプログラムのVL53L0X設定である高速測定モードでは高速に測定するため若干精度が犠牲になります。そのため測距時に1~2cmの誤差が発生します。余裕を見込んだ距離判定をしましょう
- 測定不能時の処理
 - 測定結果で8190前後の値はレーザーの反射が得られない場合の値です。つまりは測定範囲内に障害物を検知できなかった場合の値になります

Exercise-3.1

RumiCarでの開発にあたっての考慮事項(続き)

- 地面等の検知

- センサのレーザーは円錐形に照射されます。
RumiCarは小さいので車上のセンサ搭載位置が低く円錐形に照射されたレーザーはいずれ距離測定限界より先に地面や床等を捉えてしまいます。この距離はRumiCarモデル32型でだいたい30cm前後です。これらより障害物としての検知距離は30cm以下としてください

- 急停止処理

- 走行状態から停止や後進命令を出しても慣性（勢い）が付いているため直ぐには停止できません。できれば上手な減速や停止の方法を考慮しましょう

Exercise-3.1

プログラミング上の条件まとめ

項目	条件
使用センサ	中央センサ(Sensor1)のみ使用する
挙動	設定距離内に障害物を検知したら停止する
設定距離	10cmとする。ただし誤差を考え実範囲は $10 \pm 2\text{cm}$ とする
信頼距離	30cm以内とする
測定不能時の値	8190、但し誤差を含むことがあるので注意

Exercise-3.1

開発時のちょっと工夫

- 判断条件を単純に障害物がない、とのロジックとしてしまうと前方に障害物がない場合はRumiCarがどこまででも走って行ってしまいます。これだとうっかり机の上などにRumiCarをおいてしまうとRumiCarが机から飛び降りてしまいます。机の上で動作テストなどをする可能性がある場合においては安全も考えて例えば30cm以内に障害物がある場合のみ走るなどと条件を追加するなど工夫すると良いでしょう。開発時にRumiCarをうっかり破損させてしまう危険性を低減できます

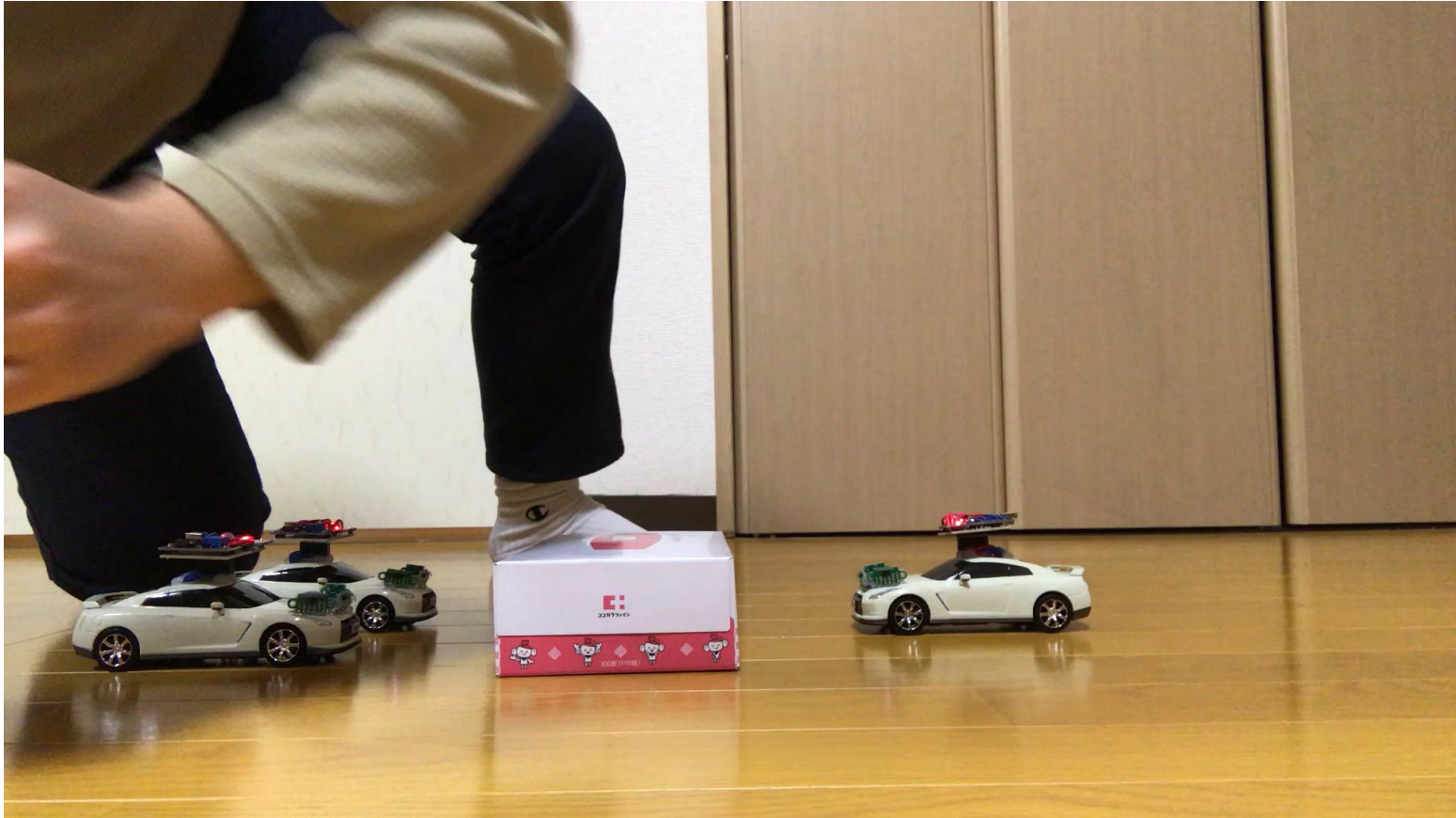
Exercise-3.1 https://youtu.be/95pc_4Wf14U

安全に停止する車（動画）



Exercise-3.1 https://youtu.be/95pc_4Wf14U

安全に停止する車 (動画)



Exercise-3.1

安全に停止する車(準備)

- RumiCarの車体の電源を**オフ**にします
- CMをRumiCarの車体より取り外してください
- CMにUSBケーブルを接続します
- USBケーブルをパソコンに接続します
- ファイル”Exercise-3.1”を開きます

Exercise-3.1

安全に停止する車(プログラム)

```
210
211 void loop()
212 {
213   int ispeed = 255;
214   int idist1;
215   idist1=sensor1.readRangeSingleMillimeters();
216   if ( idist1 < 300 ){
217     if ( idist1 > 120 ){
218       RC_drive(FORWARD,ispeed);
219     }else if (idist1 < 80){
220       RC_drive(REVERSE,ispeed);;
221     }else{
222       RC_drive(BRAKE,ispeed);
223     }
224   }else{
225     RC_drive(FREE,ispeed);
226   }
227 }
```

- 障害物が30cm以内にあるか？
- (無い場合は何もしない)
- (ある場合は次を実行)
 - 障害物が12cmを超える->前進
 - 障害物が8cm未満->後進
 - 8cm以上12cm以下->ブレーキ

繰り返す

Exercise-3.1

安全に停止する車(実走試験)

- コンパイルしプログラムをCMへ書きこみます
- CMからUSBケーブルを取り外します
- CMをRumiCarの車体に取り付けます
- RumiCar車体の電源をオンにしてください
- 手を中央センサーに近づけたり話したりしてみましょう。RumiCarは予想通りの動きをしていますか？
- RumiCarをそっと机に置いてみましょう

Exercise-3.2

市街地を走る車

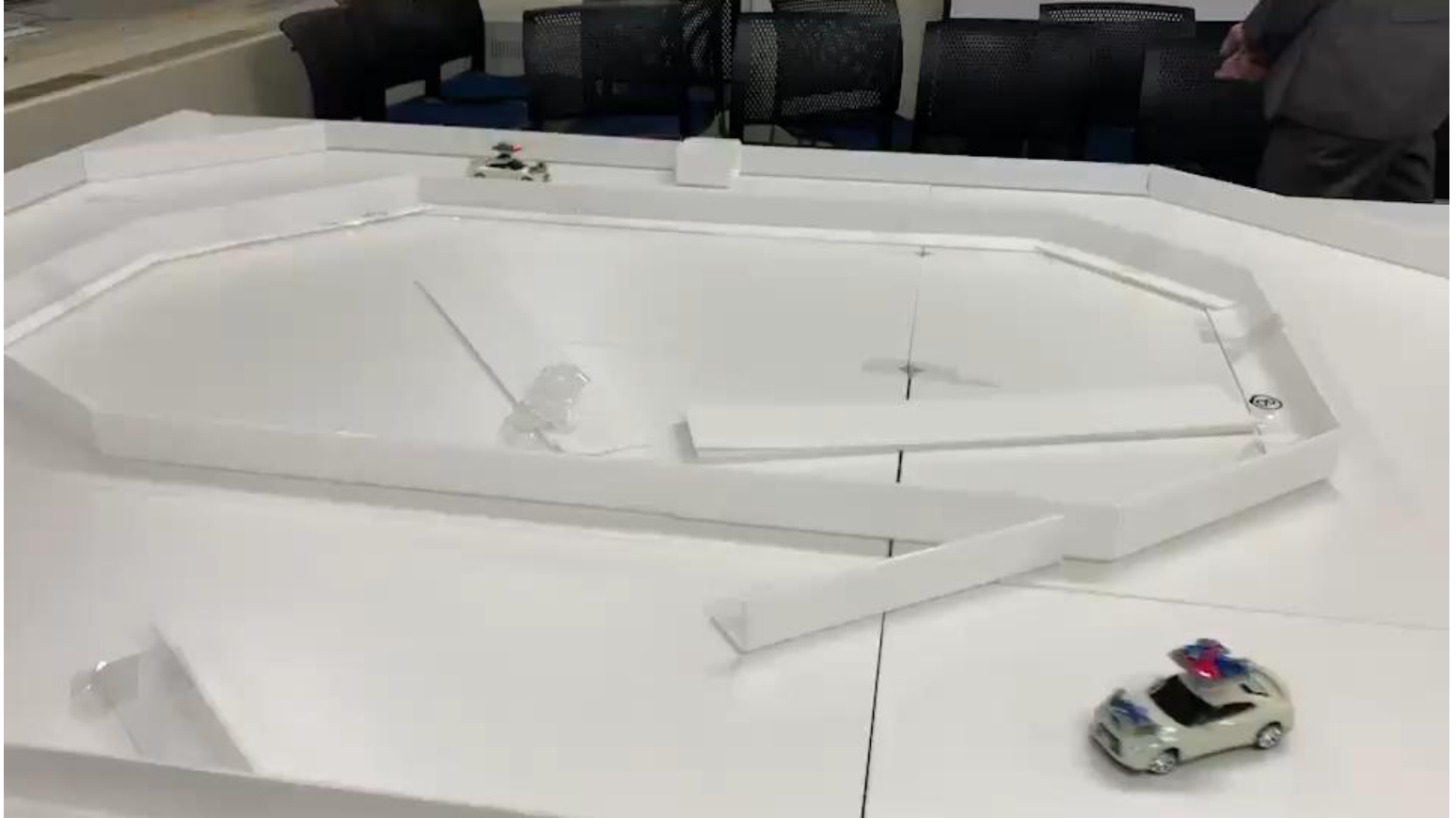
Exercise-3.2

市街地を走る車

- 単純な条件として具体的に下記を仮定します
 - 市街地に見立てた、道路の両側に壁がある道路を走ります
 - 両側の壁の距離を測って道路の中心を走ります
 - 先行車の例えば**10cm**手前で停車、先行車がなくなったら再度前進再開します

Exercise-3.2 <https://youtu.be/jyq4Ph0mCDM>

市街地を走行する車（動画）



Exercise-3.2

市街地を走行する車(準備)

- RumiCarの車体の電源を**オフ**にします
- CMをRumiCarの車体より取り外してください
- CMにUSBケーブルを接続します
- USBケーブルをパソコンに接続します
- ファイル”Exercise-3.2”を開きます

Exercise-3.2

市街地を走行する車(プログラム)

```
210
211 void loop()
212 {
213
214 int ibound =250;
215 int s0, s1, s2;
216 s0=sensor0.readRangeSingleMillimeters();
217 s1=sensor1.readRangeSingleMillimeters();
218 s2=sensor2.readRangeSingleMillimeters();
219
220 if(s1<100){
221   RC_drive(REVERSE,150);
222 }else if (s1<150){
223   RC_drive(FORWARD,150);
224 }else if (s1<250){
225   RC_drive(FORWARD,200);
226 }else{
227   RC_drive(FORWARD,255);
228 }
229 if(s0>s2){
230   RC_steer(LEFT);
231 }else{
232   RC_steer(RIGHT);
233 }
234 }
```

- 速度のパラメータ（150や200、255）を変更してみよう！
- 距離のパラメータ（100や150、250）を変更してみよう！
- どうなるかな？？

- 障害物が10cm未満->ゆっくり後進
- 障害物が15cm未満->ゆっくり前進
- 障害物が25cm未満->ちょっと速めに前進
- 上記以外は全速前進

- 左右の壁の距離を比較して
- 壁から近い方から離れる（壁から遠い方へハンドルを切る）

Exercise-3.2

市街地を走行する車(実走試験)

- コンパイルしプログラムをCMへ書きこみます
- CMからUSBケーブルを取り外します
- CMをRumiCarの車体に取り付けます
- RumiCar車体の電源をオンにしてください
- RumiCarをそっとコースに置いてみましょう

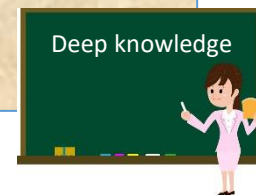
Deep knowledge



Deep knowledge

ラズパイサンプルプログラムに関して
石垣 翔子 氏

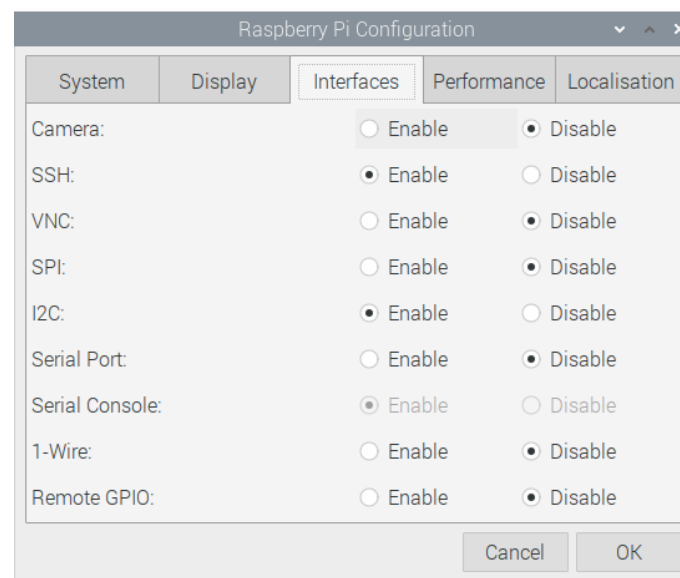
ラズパイサンプルプログラムに関して



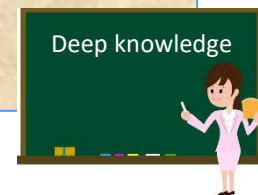
- PWM制御を含め全てのGPIO制御に
[pigpioライブラリ](#)を使用しています。
プログラム実行前にpigpiodを起動してください。

起動コマンド : `sudo pigpiod`

- 測距モジュールはI2Cを使用するため、
事前にI2Cを有効にしてください。



ラズパイサンプルプログラムに関して



- ソフトウェアPWMとハードウェアPWM
 - ラズパイで独立して制御できるハードウェアPWMは2つ
12/32番ピンと33/35番ピンがHardware PWMとして使用可能
 - ソフトウェアPWMは全てのGPIOピンで使用できます
 - 急停止など正確な制御を要する走行用モータにハードウェアPWMを使用します

ラズパイサンプルプログラムに関して



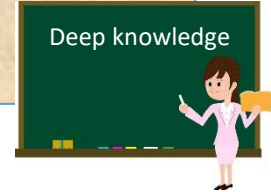
VL53L0Xのプログラム実行に必要なライブラリのインストール方法

- GitHubに公開されているライブラリを使用します
- 1. `git clone https://github.com/johnbryanmoore/VL53L0X_rasp_python`
- 2. cloneしたVL53L0X_rasp_pythonフォルダ内でmakeを実行

注意点

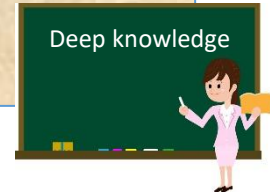
- サンプルプログラムは上記レポジトリが/home/pi/Documentsにcloneされていることを前提としています。必要に応じてVL53L0X.py内の共通ライブラリのパスを変更してください。
- VL53L0XのVccにラズパイの3.3V出力以外を使用する場合はラズパイのGNDピンをVL53L0Xと共通のGNDに接続してください

Azure IoT Hub



- IoTデバイス（ラズパイなど）とAzureの双方間通信
- 走行データの可視化や遠隔操作が可能に
- Azure Machine Learningなどの他のAzureサービスとの連携

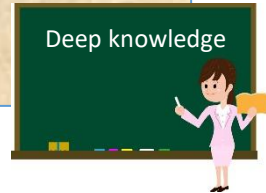
device-to-cloudメッセージ



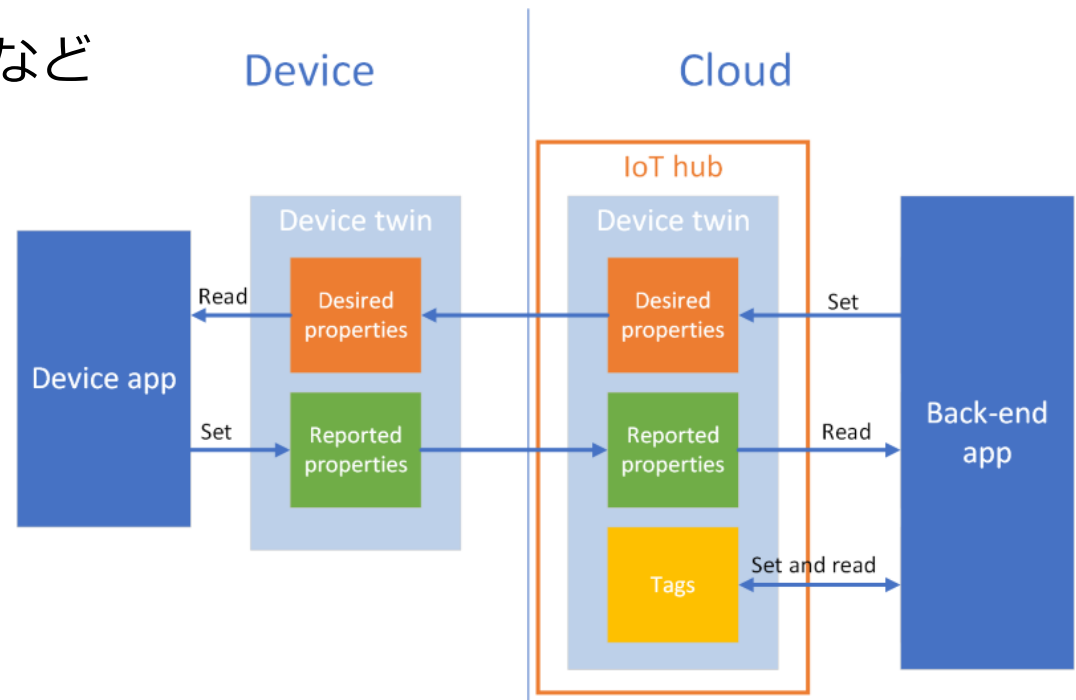
- センサーデータの送信と監視
- 最大256-KBのデータまで送信可能

```
ShokonoMacBook-Pro:~ $ az iot hub monitor-events --hub-name raspi-ishigaki --device-id raspi
Starting event monitor, filtering on device: raspi, use ctrl-c to stop...
{
  "event": {
    "origin": "raspi",
    "interface": "",
    "payload": "{\"distance\\\": 424, \\\"control\\\": FREE}"
  }
}
{
  "event": {
    "origin": "raspi",
    "interface": "",
    "payload": "{\"distance\\\": 422, \\\"control\\\": FREE}"
  }
}
{
  "event": {
    "origin": "raspi",
    "interface": "",
    "payload": "{\"distance\\\": 422, \\\"control\\\": FREE}"
  }
}
```

desired properties



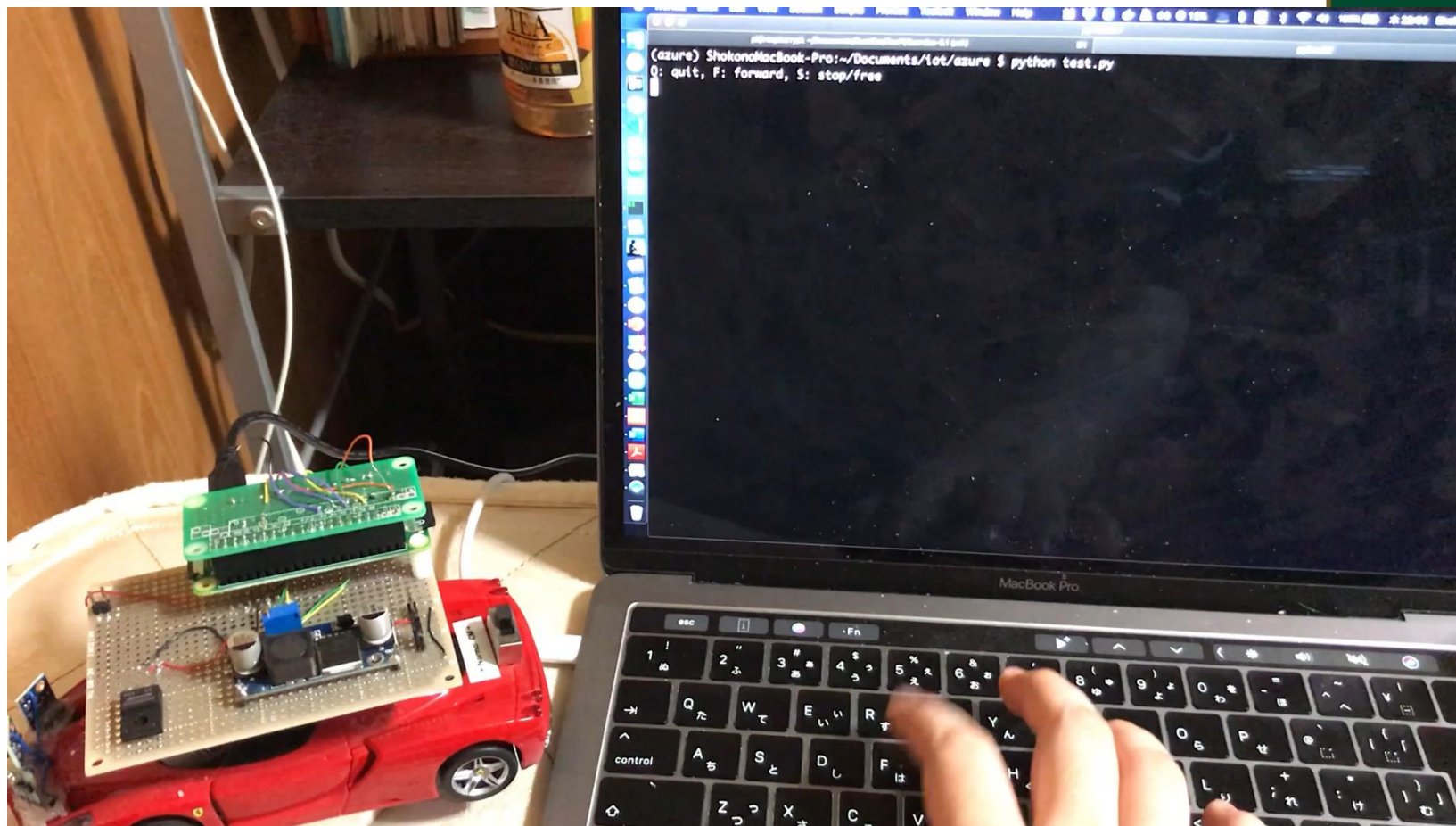
- クラウドからデバイスへの通知
- 実行開始や停止の制御命令など



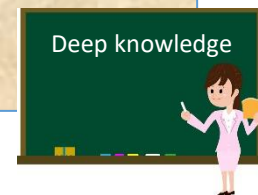
<https://docs.microsoft.com/en-us/azure/iot-hub/tutorial-device-twins>

動作動画（動画）

Deep knowledge



参考情報



- Azure IoT Hub のチュートリアル
 - <https://docs.microsoft.com/ja-jp/azure/iot-hub/quickstart-send-telemetry-cli>
- ソースコード（ラズパイ）
 - <https://github.com/RumiCar-group/RumiCar/tree/master/RasPi/misc>
 - クラウドへの送信 : azure_d2c.py
 - Desired property : azure_property.py（ラズパイ側）、azure_service.py（PCなど制御が側）

※Azure IoT Hubの作成が必要です。

付録

I2Cアドレス

- RumiCarではI2C通信を利用します。現在使用中のアドレスです

センサ			アドレス	設定方法
VL53L0X (レーザ測距モジュール)	Sensor0	左	0x20	ソフトウェアにより変更
	Sensor1	中央	0x21	
	Sensor2	右	0x22	
	Sensor3	後部	0x52	CMの後部に設置 ソフトウェアにより変更不可
BMX055 (9軸センサ)	加速度センサ		0x19	基板上にてジャンパ設定
	ジャイロセンサ		0x69	
	磁気センサ		0x13	

分野別開発協力者

氏名	内容
片岡 秀明	• RumiCar車体作製 • CM開発
大栄 豊	• センサー基板設計開発 • Arduino Nano版CM基板設計開発
石垣 翔子	• ラズパイ版CM開発 • Pythonプログラム開発 • Microsoft Azure IoT Hub連携
稲玉 繁樹	• モータ低速回転制御 • ESP32版基板設計開発 • M5StickC版基板設計開発 • センサー基板設計開発
高瀬 英希	• ライブラリ化

イベント履歴

	日付	イベント	開催場所	備考
1	2017年8月5-6日	MakerFaireTokyo2017	東京ビックサイト	
2	2018年5月18-20日	MakerFaireBayArea2018	San Mateo, USA	
3	2018年8月4-5日	MakerfaireTokyo2018	東京ビックサイト	
4	2019年3月15日	Hello Tomorrow Global Challenge	Paris, France	投資家イベント
5	2019年8月3-4日	MakerFiareTokyo2019	東京ビックサイト	
6	2019年11月30日	IoT ALGYANつくるよ2！	情報科学専門学校	
7	2020年4月18日	IoT ALGYAN RumiCarハンズオン中継	オンライン	
8	2020年4月25日	IoT ALGYAN RumiCarハンズオン中継	オンライン	
9	2020年6月6日	IoT ALGYAN RumiCarハンズオン	オンライン	実機貸出でのオンライン

イベント履歴

	日付	イベント	開催場所	備考
10	2020年6月20日	【リアル】マイRumiCar試走会！RumiCar開発部メンバーと一緒に、自慢の愛機を走らせよう！	川崎市麻生区	
11	2020年8月10日	RumiCar実機ハンズオン&走行会	横浜市開港記念会館	
12	2020年8月30日	RumiCar走行会	麻生文化センター	
13	2020年11月13日	～RumiCar を使って自動運転車の制御の基本を学ぼう～ 第一回 部品確認、ハードウェアの作成	Zoom	主催:一般社団法人 広島県情報産業協会 技術委員会
14	2020年12月4日	～RumiCar を使って自動運転車の制御の基本を学ぼう～ 第二回 ハードウェア作成の続き、基本的なソフトウェアのテスト		協賛:RumiCar開発部

イベント履歴

	日付	イベント	開催場所	備考
15	2021年1月10日	第15回イベント:RumiCarハンズオン&走行会(300人用会場で開催します!)	川崎市麻生区	
16	2021年9月26日	RumiCar会議(conference)	オンライン	
17	2021年11月28日	RumiCar走行会(良いにゃの日)	麻生文化センター	
18	2021年12月12日	RumiCar会議(conference)12/12	オンライン	Youtubeライブ
	2022年4月29日	メイカーズフリマ	技研ベース	
19	2022年8月7日	第17回イベント：RumiCar試走会	東京都品川区 大崎第二区民集会所	イベント座長 井上氏
20	2022年9月3-4日	Maker Faire Tokyo2022	東京ビッグサイト	
21	2022年10月23日から27日	第一回RumiCar画廊(個展)	隅田公園リバーサイドギャラリー	

イベント履歴

	日付	イベント	開催場所	備考
22	2022年11月3日、 5日、6日、12日、 13日	第二回RumiCar画廊(個展)	隅田公園リバーサ イドギャラリー	
23	2022年9月3-4日	Maker Faire Tokyo2022	東京ビッグサイト	
24	2022年12月31日	コミックマーケット101	東京ビッグサイト	

メディア掲載

	メディア	日付	内容	ページ
1	I/O 第47巻7月号 通巻549号	2022年7月号	100円ロボット部!	118-119
2	I/O 第47巻8月号 通関550号	2022年8月号	RumiCar	10-13
3	「Raspberry Pi」から広がる電子工作の世界	2022年11月30日発売		

提携・協業等

	メディア	日付	内容
1	R2		
2			
3			

最後に

RumiCar関連情報

より良く楽しんでいただくために

- RumiCar開発部では多くの方々に楽しんでいただくために標準化が大切と考えています
- 標準化がなされていると車体やコンピュータモジュールのハードウェアの互換性が向上し、プログラムの再利用性が高まります
- これらにより、多くの人が変更の手間を掛けずに機能向上、機能追加、高機能化ができるようになります
- 多くの人により楽しんで頂くためにコンピュータモジュールのピンアサインやピン配置はRumiCar標準に準拠いただき開発をお願いします

RumiCar関連情報

- RumiCar開発部Webページ
 - <https://www.rumicar.com/>
- Facebook RumiCarグループ
 - <https://www.facebook.com/groups/rumicar>
- RumiCar開発部 連絡先メールアドレス
 - info@RumiCar.com
- RumiCar YouTube
 - https://www.youtube.com/channel/UCVg3CBSVBcc_00FdC6q2wDg
- GitHub
 - <https://github.com/RumiCar-group/RumiCar>



RumiCarハンズオン！に
ご参加頂きましてどうも
ありがとうございました

これで終了です